

Prácticas de Tarjetas Gráficas y Aceleradores

Agustín Fernández

Daniel Jiménez

Departament d'Arquitectura de Computadors

Índice

1. Sesión 1 – Primeros pasos con CUDA	5
1.1. El servidor	5
1.2. Conexión, Cuentas y Passwords	5
1.3. Uso de las colas	6
1.4. Antes de empezar	7
1.5. ScanDevices	7
1.6. Una ejecución de Scandevise	9
1.7. SaxpyP	10
1.8. Profiling	11
1.9. Control de Errores en CUDA	12
1.10. Recursos Adicionales	13
2. Sesión 2 – Puzzles	14
2.1. Introducción	14
2.2. Grids, Bloques y Threads	14
2.3. Puzzle 1D	15
2.4. Puzzle 2D	16
2.5. Puzzle 3D	16
2.6. Profiling	17
2.7. Consideraciones Finales	18
3. Sesión 3 – Reducciones	19
3.1. Introducción	19
3.2. Kernel 01 (main01.cu)	20
3.3. Kernel 02 (main02.cu)	21
3.4. Kernel 03 (main03.cu)	22
3.5. Kernel 04 (main04.cu)	22
3.6. Kernel 05 (main05.cu)	23
3.7. Kernel 06 (main06.cu)	24
3.8. Kernel 07 (main07.cu) [ÚLTIMO TEST]	25

3.9. Profiling	26
3.10. Consideraciones Finales	26
4. Sesión 4 – Producto de Matrices	27
4.1. Introducción	27
4.2. Kernel 00 (MM00.cu)	27
4.3. Kernel 01 (MM01.cu)	29
4.4. Kernel 10 (MM10.cu)	30
4.5. Kernel 11 (MM11.cu)	31
4.6. Pinned Memory	31
4.7. Otras cosas a probar	31
4.8. Un desafío	32
4.9. Consideraciones Finales	32
5. Sesión 5 – multi GPU	33
5.1. Introducción	33
5.2. KERNEL MultiGPU	34
5.3. Tomando Tiempos	35
5.4. cudaMemcpyAsync	36
5.5. CUDA Samples y Documentación	36
5.6. Consideraciones Finales	37
6. Sesión 6 – Streams	38
6.1. Kernel 00 (stream00.cu)	38
6.2. Hablemos de Streams	39
6.3. Kernel 01 (stream01.cu)	40
6.4. Kernel 02 (stream02.cu)	41
6.5. Algunos comentarios respecto a la toma de tiempos	41
6.6. Consideraciones Finales	42
7. Proyecto – Algoritmo de Strassen	43
7.1. Introducción	43
7.2. El algoritmo	43

7.3. Implementación CUDA en 1 GPU	44
7.4. Implementación CUDA en varias GPUs	45
7.5. Resultados esperados	46
8. Proyecto – Histogram Equalization	47
8.1. Introducción	47
8.2. Algoritmo	47
8.2.1. Tabla de Equalización	47
8.3. Implementación CUDA en 1 GPU	48
8.4. Implementación CUDA en varias GPUs	48
8.5. Resultados esperados	48
9. Proyecto – Heat: Jacobi o Gauss-Seidel	50
9.1. Introducción	50
9.2. Algoritmo Jacobi	50
9.3. Algoritmo Gauss-Seidel	50
9.4. Implementación CUDA en 1 GPU	51
9.5. Implementación CUDA en varias GPUs	52
9.6. Resultados esperados	52
10. Proyecto – Mandelbrot	54
10.1. Introducción	54
10.2. Algoritmo de Mandelbrot	54
10.3. Implementación CUDA en 1 GPU	55
10.4. Implementación CUDA usando Paralelismo dinámico	55
10.5. Resultados esperados	55
11. Python y CUDA	56
11.1. Objetivo del proyecto	56
11.2. Punto de Partida	56
Bibliografía	57

1 SESIÓN 1 – PRIMEROS PASOS CON CUDA

En esta primera sesión describiremos el entorno de trabajo, como acceder a los recursos, y los pasos necesarios para compilar y ejecutar un programa en **CUDA**.

Como ejemplo utilizaremos dos programas muy conocidos de **CUDA**: Scandevices y Saxpy. Acabaremos la sesión utilizando la herramienta de profiling que proporciona NVIDIA: nvprof.

1.1 El servidor

Utilizaremos el mismo servidor que usabais en PAR (Paralelismo): boada. Este servidor empezó con 4 nodos y en cada nodo 2 procesadores Intel Xeon E5645 de 6 cores con 24GB de memoria DRR3 1333 ECC. El servidor ha ido evolucionando, se ha actualizado, ha crecido, ... pero el nombre se mantiene. Actualmente boada tiene 47 nodos, de los cuales 32 son sistemas embedded.

En este servidor tenemos un nodo (boada-10) con 2 procesadores Intel Xeon 4314 2.4 GHz de 16 cores por procesador y un total de 128 GB de memoria DDR4. Además, en este nodo hemos instalado 4 tarjetas NVIDIA RTX 3080. Cada RTX 3080 dispone de 8.704 cores de **CUDA** con 10 GB de memoria GDDR6X (ancho de banda de 760,3 GB/s) y tiene una potencia de cálculo de 29,77 TFLOPs en FP32 (465,1 GFLOPs en FP64).

En otoño de 2024 instalamos una tarjeta RTX 4090. Es la tarjeta gráfica discreta más potente que se podía comprar a finales de 2024. Dispone de 16.384 cores de **CUDA** con 24 GB de memoria GDDR6x (ancho de banda de 1.008 GB/s) y tiene una potencia de cálculo de 82,58 TFLOPs en FP32 (1,29 TFLOPs en FP64).

Normalmente trabajaréis en uno de los nodos interactivos de boada (edición, compilación) y cuando necesitéis trabajar con las GPUs, lo haréis utilizando el sistema de colas. **No se podrá trabajar de forma interactiva con las GPUs.**

1.2 Conexión, Cuentas y Passwords

Para trabajar con el servidor necesitáis trabajar desde Linux. Para establecer la conexión con el servidor hay que usar *secure shell*¹:

```
ssh -X username@boada.ac.upc.edu
```

La opción **-X** será necesaria cuando necesitéis abrir una ventana remota en vuestro escritorio. El *username* y *password* os lo proporcionaremos en la primera sesión de laboratorio. Si utilizáis un Mac hay que usar la opción **-Y**, e instalar previamente el programa **Xquartz**.

¹**¡MUY IMPORTANTE!** Si os conectáis desde casa, tenéis que hacerlo desde la VPN de la UPC, utilizando el servicio UPCLink. Los detalles de cómo conectaros los podéis encontrar en: <https://serveistic.upc.edu/ca/upclink>.

Una de las primeras cosas que tendréis que hacer es cambiar el *password*, lo podéis hacer usando el siguiente comando:

```
ssh -t username@boada.ac.upc.edu passwd
```

El cambio de *password* parece muy lioso, pero tiene su lógica. Supongamos que tenéis como *username* / *password*: `cudaXXXX` / `12345678`, y queréis que el nuevo *password* sea `abcdefgh`. Hay que hacer lo siguiente:

```
> ssh -t cudaXX@boada.ac.upc.edu passwd
cudaXXXX@boada.ac.upc.edu's password: 12345678
(current) UNIX password: 12345678
Enter new UNIX password: abcdefgh
Retype new UNIX password: abcdefgh
```

Hemos puesto en **ROJO** lo que tenéis que escribir vosotros. Fijaos que hay que poner el *password* viejo 2 veces, y el nuevo otras 2. Repito, lioso, pero tiene lógica (el primer *password* 12345 es para entrar en boada, el segundo *password* 12345 es el que os piden por el cambio de *password*, el tercero es el nuevo y el cuarto la confirmación).

Cuando entréis por primera vez, vuestra zona está prácticamente vacía. Podéis organizarla como más os convenga. Recordad haceros una copia de todo lo que hagáis (de esta máquina no hacemos *backup*). Para empezar, sólo nos queda por saber cómo transferir ficheros entre nuestra máquina y boada. Para copiar datos entre las dos máquinas usaremos `scp`²:

```
scp file username@boada.ac.upc.edu:.
scp username@boada.ac.upc.edu:file .
```

El comando `scp` lo tenemos que ejecutar **siempre** en nuestra máquina local y **nunca** en boada.

En todas las sesiones colgaremos un fichero tal como éste: `Sesion01.tar`. Antes de empezar hay que desempaquetar el fichero con el siguiente comando:

```
tar xvf Sesion01.tar
```

1.3 Uso de las colas

El comando a utilizar para mandar un trabajo a la cola de ejecución es:

```
sbatch job.sh
```

El contenido del fichero `job.sh`³ podría ser el siguiente:

²¡MUY IMPORTANTE! Si no os funciona el `scp` es posible que necesitéis poner la opción "-O": `scp -O`, y el resto igual.

³En todas las sesiones os daremos una versión operativa inicial del `job.sh`

```
#!/bin/bash

### Directivas para el gestor de colas
#SBATCH --job-name=deviceQuery
#SBATCH -D .
#SBATCH --output=submit-deviceQuery.o%j
#SBATCH --error=submit-deviceQuery.e%j
#SBATCH -A cuda
#SBATCH -p cuda
#SBATCH --qos=cuda3080
#SBATCH --gres=gpu:rtx3080:1

export PATH=/Soft/cuda/12.2.2/bin:$PATH

./deviceQuery.exe
```

Al acabar la ejecución se generan 2 ficheros: `deviceQuery.oXXXX` y `deviceQuery.eXXXX`. En el primero está la salida de `deviceQuery.exe`, en el segundo los posibles errores y la información de *profiling* cuando la pidáis. `XXXX` es un contador que varía en cada ejecución.

Otros comandos útiles para tratar con las colas son:

- `squeue`, permite comprobar el estado de las colas
- `scancel jobID`, permite eliminar un trabajo del sistema de colas
- `scontrol show node boada-10`, permite ver el estado del nodo boada-10
- `sinfo`, permite ver el estado general todo el clúster

Para obtener más detalles de estos comandos utilizad el [man](#).

1.4 Antes de empezar

Todo lo necesario para la sesión 01 lo encontraréis en el fichero `Sesion01.tar`. En todas las sesiones tendréis un fichero similar. Los ficheros con la extensión `.tar` hay que desempaquetarlos con el siguiente comando:

```
tar xvf Sesion01.tar
```

Recordad que previamente hay que hacer un `scp` de `Sesion01.tar` de la máquina local a boada.

1.5 ScanDevices

Este test sólo se ha de compilar y ejecutar. Para ejecutar un programa en **CUDA** en el clúster hay que utilizar el sistema de colas. En nuestro caso para compilar y ejecutar:

```
make
sbatch job.sh
```

En todas las sesiones incluiremos el [Makefile](#) correspondiente, así como el *script* para lanzar la ejecución a una cola, especificada en el fichero [job.sh](#).

Al ejecutar este test sabremos cuántas GPUs tenemos instaladas en el servidor y para cada una de ellas (pueden ser diferentes) información detallada de sus características. La información de cada GPU es un listado parecido a éste:

```
Device 0: ---
Major revision number:      ---
Minor revision number:      ---
Total amount of global memory:  --- bytes
Number of multiprocessors:    ---
Number of cores:             ---
Total amount of constant memory: --- bytes
Total amount of shared memory per block:  --- bytes
Total number of registers available per block: ---
Warp size:                   ---
Maximum number of threads per block:      ---
Maximum sizes of each dimension of a block:  --- x --- x ---
Maximum sizes of each dimension of a grid:  --- x --- x ---
Maximum memory pitch:         --- bytes
Texture alignment:            --- bytes
Clock rate:                   --- GHz
Concurrent copy and execution: ---
...
```

Algunos de estos valores (básicamente los marcados en **rojo**) los deberemos tener en cuenta al programar en **CUDA** con esa GPU. Por ejemplo, si lanzamos un kernel con más *threads* de los que soporta la GPU, el kernel no funcionará (dará un error).

Cuando instalamos una GPU + **CUDA** este es el primer test que hay que correr. Primero para comprobar que la GPU funciona, y luego para obtener estos parámetros.

Ahora, si miráis el `job.sh`, hay un par de parámetros muy interesantes:

```
#SBATCH --qos=cuda3080
#SBATCH --gres=gpu:rtx3080:1
```

Básicamente, le estamos diciendo al gestor que vamos a utilizar la cola `cuda`, servicio `--qos=cuda3080`, que implica el uso de las RTX 3080, y con estos recursos `--gres=gpu:rtx3080:1`, que indica cuantas GPUs queremos utilizar, en este caso sólo 1. Podemos cambiarlo de la siguiente forma:

```
#SBATCH --qos=cuda3080
#SBATCH --gres=gpu:rtx3080:4
```


Y volver a ejecutar el programa de otra vez. ¿Qué ha ocurrido?

Otro cambio que podemos hacer es el siguiente:

```
#SBATCH --qos=cuda4090
#SBATCH --gres=gpu:rtx4090:1
```

En este caso estaremos utilizando la RTX 4090 (en este caso sólo tenemos 1). Recordad estas 3 configuraciones, porque las utilizaremos muchas veces a lo largo del curso.

Es un buen momento para consultar el manual online de **CUDA**. La información que ofrece este test se obtiene llamando a la función `cudaGetDeviceProperties()`. Si consultamos la siguiente página: docs.nvidia.com/cuda/cuda-runtime-api/ y buscamos esta rutina, veremos la gran cantidad de información que podemos obtener con ella. Para acceder a la documentación de NVIDIA, es necesario tener un usuario registrado. Os recomendamos que os déis de alta en <https://developer.nvidia.com> para poder acceder a todos los recursos de **CUDA** que ofrece NVIDIA.

1.6 Una ejecución de Scandevic

Lo que mostramos a continuación es la ejecución de esta función en otro servidor. En este servidor disponemos de 2 tarjetas gráficas: una GTX 1080ti y una Tesla K40. Es muy interesante comprobar las diferencias entre estos dos tarjetas, la GTX de la Familia Pascal y la K40 de la familia Kepler con las RTX 3080 y la nueva RTX 4090.

```
CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 2 CUDA Capable device(s)

Device 0: "GeForce GTX 1080 Ti"
  CUDA Driver Version / Runtime Version      8.0 / 8.0
  CUDA Capability Major/Minor version number: 6.1
  Total amount of global memory:              11158 MBytes (11700207616 bytes)
  (28) Multiprocessors, (128) CUDA Cores/MP: 3584 CUDA Cores
  GPU Max Clock rate:                        1582 MHz (1.58 GHz)
  Memory Clock rate:                          5505 Mhz
  Memory Bus Width:                           352-bit
  L2 Cache Size:                             2883584 bytes
  Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768), 2048 layers
  Total amount of constant memory:            65536 bytes
  Total amount of shared memory per block:     49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                  32
  Maximum number of threads per multiprocessor: 2048
  Maximum number of threads per block:         1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                      2147483647 bytes
  Texture alignment:                          512 bytes
  Concurrent copy and kernel execution:       Yes with 2 copy engine(s)
  Run time limit on kernels:                   Yes
  Integrated GPU sharing Host Memory:          No
  Support host page-locked memory mapping:    Yes
  Alignment requirement for Surfaces:         Yes
  Device has ECC support:                     Disabled
  Device supports Unified Addressing (UVA):    Yes
  Device PCI Domain ID / Bus ID / location ID: 0 / 5 / 0
```

```

Device 1: "Tesla K40c"
  CUDA Driver Version / Runtime Version      8.0 / 8.0
  CUDA Capability Major/Minor version number: 3.5
  Total amount of global memory:             11440 MBytes (11995578368 bytes)
  (15) Multiprocessors, (192) CUDA Cores/MP: 2880 CUDA Cores
  GPU Max Clock rate:                        745 MHz (0.75 GHz)
  Memory Clock rate:                         3004 Mhz
  Memory Bus Width:                          384-bit
  L2 Cache Size:                            1572864 bytes
  Maximum Layered 1D Texture Size, (num) layers 1D=(16384), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers 2D=(16384, 16384), 2048 layers
  Total amount of constant memory:           65536 bytes
  Total amount of shared memory per block:    49152 bytes
  Total number of registers available per block: 65536
  Warp size:                                32
  Maximum number of threads per multiprocessor: 2048
  Maximum number of threads per block:       1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                      2147483647 bytes
  Texture alignment:                         512 bytes
  Concurrent copy and kernel execution:       Yes with 2 copy engine(s)
  Run time limit on kernels:                  No
  Integrated GPU sharing Host Memory:         No
  Support host page-locked memory mapping:    Yes
  Alignment requirement for Surfaces:         Yes
  Device has ECC support:                     Enabled
  Device supports Unified Addressing (UVA):   Yes
  Device PCI Domain ID / Bus ID / location ID: 0 / 9 / 0
  Compute Mode:

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 8.0,
CUDA Runtime Version = 8.0, NumDevs = 2,
Device0 = GeForce GTX 1080 Ti, Device1 = Tesla K40c
Result = PASS

```

1.7 SaxpyP

En este test se calcula la operación **Saxpy** que vimos como ejemplo en las clases de teoría. Para compilar y ejecutar usaremos los ficheros que hay en el directorio. En nuestro caso:

```

make
sbatch job.sh

```

Si miráis el fichero ([main.cu](#)), veréis que tiene una estructura similar a la que hemos visto en clase de teoría con algunos añadidos. Lo más novedoso es la forma en que medimos los tiempos de ejecución. Teniendo en cuenta que, la ejecución de determinadas operaciones en la GPU es asíncrona con respecto a la CPU, no es posible usar las mismas rutinas que usaríamos en un programa convencional para medir el tiempo de ejecución.

Tenemos que usar los eventos de **CUDA** ([cudaEvent_t](#)). Lo que hay que hacer es registrar los eventos necesarios entre el código a medir:

```

cudaEventRecord(E0, 0); cudaEventSynchronize(E0);
// Código a medir
cudaEventRecord(E1, 0); cudaEventSynchronize(E1);
cudaEventElapsedTime(&tiempo, E0, E1);

```

La rutina `cudaEventSynchronize()` provocará que el programa espere hasta que el evento quede registrado en la GPU. Esto es imprescindible cuando trabajamos con operaciones asíncronas. La rutina `cudaEventElapsedTime()` calcula el tiempo transcurrido (en ms) entre el registro de los 2 eventos.

Con este test se pueden realizar muchas pruebas. Os enumeramos algunas:

1. Compilad, ejecutad y comprobad que funciona correctamente.
2. Calculad los MFLOPS contando sólo el kernel.
3. Calculad los MFLOPS teniendo contando también las transferencias de datos.
4. Calculad el ancho de banda de las transferencias CPU → GPU, y GPU → CPU.
5. Calculad los mismos anchos de banda, pero ahora utilizando memoria “pinned”. El código necesario ya está escrito en un comentario.
6. La rutina que comprueba que el resultado es correcto, hace este test de error: $(\text{abs}(a-b)/a > E)$. Cambiad el test de error por uno de igualdad ($a \neq b$). ¿Qué ocurre en este caso?
7. Modificad `job.sh` para cambiar el tamaño del problema y el número de *threads* que se utiliza. No hay que modificar el código, sólo hay que invocar la ejecución así:

```
./SaxpyP.exe 16777216 1024
```

Siendo 16777216 el tamaño del problema (N) y 1024 en número de *threads* (nThreads).

Jugad con el número de *threads* a ver qué pasa.

1.8 Profiling

Como en muchos otros entornos, tenemos herramientas para analizar el rendimiento de una aplicación **CUDA**. La herramienta más simple es `nvprof`. La forma de invocarla es muy simple:

```
nsys nvprof --print-gpu-summary ./SaxpyP.exe
```

El resultado del profiling se escribirá en el fichero de error que nos devuelve la cola `cuda`. Parte de la información que ofrece `nvprof` es la siguiente:

- Estadísticas de ejecución de los kernels.
- Estadísticas de las operaciones de comunicación CPU – GPU.
- El tiempo que hemos gastado en cada una de las rutinas de **CUDA**.
- ...

Para más detalles se puede consultar docs.nvidia.com/cuda/profiler-users-guide.

1.9 Control de Errores en CUDA

En este último test hemos incluido el código necesario para comprobar errores. En **CUDA**, todas las llamadas devuelven un código de error (a excepción de la ejecución de un kernel). El código de error es un tipo predefinido: `cudaError_t`. También existe una función **CUDA** que devuelve el código del último error:

```
cudaError_t cudaGetLastError(void)
```

Para saber de que error se trata, disponemos de una función que devuelve un string con la descripción del error:

```
char* cudaGetErrorString(cudaError_t)
```

La rutina de error que podéis encontrar en el código es la siguiente:

```
void CheckCudaError(char sms[]) {  
    cudaError_t error;  
  
    error = cudaGetLastError();  
    if (error) printf("%s: %s\n", sms, cudaGetErrorString(error));  
}
```

Esta rutina se puede usar después de cada invocación a una rutina **CUDA**, como por ejemplo:

```
cudaMemcpy(H_y, d_y, numBytes, cudaMemcpyDeviceToHost);  
CheckCudaError((char *) "Copiar Datos Device --> Host");
```

En caso de que la rutina `cudaMemcpy` produzca un error, nos aparecerá el mensaje:

```
Copiar Datos Device --> Host: DESCRIPCION DEL ERROR
```

Para ver cómo funciona el control de errores podéis probar las siguientes cosas:

1. Sin variar el tamaño del problema, modificad el número de *threads*, siempre con valores múltiplo de 32 (p.e. 32, 64, 128, 256, 512 y 1024). ¿Hay cambios en el rendimiento?
2. Ahora probad con 1024 + 32 *threads*. No funciona. ¿Porqué?
3. Sin variar el tamaño del problema, modificad el número de *threads*, pero ahora con cualquier valor (p.e. 50, 100, 200, 500, 1000). ¿Por qué no funciona? ¿Se puede arreglar? ¿Y el rendimiento?

1.10 Recursos Adicionales

Junto con la distribución gratuita de **CUDA** que se puede obtener en la página web de nvidia podemos encontrar algunos recursos muy útiles. Estos recursos adicionales los podéis encontrar en el directorio: [/Soft/cuda/VERS⁴](#). En particular nos interesa lo siguiente:

- **Ejemplos.** En el directorio [/Soft/cuda/VERS/samples](#) encontraréis numerosos ejemplos de **CUDA**. Desde aplicaciones muy simples, hasta ejemplos avanzados. Estos ejemplos se pueden compilar fácilmente modificando los [Makefiles](#) de la sesión actual. A partir de la versión 12 de **CUDA** estos ejemplos sólo están disponibles en <https://github.com/NVIDIA/cuda-samples>.
- **Documentación.** En el directorio [/Soft/cuda/VERS/doc/pdfs](#) se podían encontrar todos los manuales de **CUDA** de la versión correspondiente: manuales de consulta, de buenas prácticas, de optimización, etc.. Este directorio existía hasta la versión 9. Para encontrar información más actualizada hay que visitar: <https://docs.nvidia.com/cuda> (ahí está la información de todas las versiones del compilador).
- **Herramientas.** En el directorio [/Soft/cuda/VERS/tools](#) encontraréis una herramienta muy útil, el fichero Excel [CUDA_Occupancy_Calculator.xls](#), cuya utilidad tendréis que investigar vosotros mismos.

⁴VERS depende de la versión de **CUDA** instalada. Nosotros estamos usando la 12.2.2.

2 SESIÓN 2 – PUZZLES

Grid, Bloques y Threads. Estos tres conceptos son fundamentales para escribir programas eficientes en **CUDA**. En esta sesión vamos a programar un conjunto de programas muy simples, pero que ilustrarán cómo particionar un problema en Bloques y *Threads* con vectores y matrices de 2 y 3 dimensiones.

2.1 Introducción

En esta sesión vamos a trabajar con ejemplos de código muy simples. El objetivo es que os familiaricéis con el particionado de cualquier problema en Bloques y *Threads*, y aprendáis a repartir los *threads* de ejecución de un kernel considerando la arquitectura y los datos del problema a resolver.

Existen situaciones en donde puede resultar interesante distribuir los datos del problema de forma análoga a la estructura de los *threads* de ejecución, que se determina al invocar un kernel. Como ejemplo de tales situaciones, se pueden mencionar aquellos kernels que realizan operaciones con matrices de 2 o 3 dimensiones.

También haremos énfasis en problemas cuyo tamaño NO es múltiplo del número de *threads* y evaluaremos si eso supone una pérdida de rendimiento.

2.2 Grids, Bloques y Threads

Durante la ejecución de un kernel podemos utilizar una serie de variables predefinidas para identificar el *thread* en ejecución:

```
dim3 threadIdx;    // numero de thread dentro del block
dim3 blockIdx;     // numero de block dentro del grid
dim3 blockDim;     // Dimensiones del block
dim3 gridDim;      // Dimensiones del grid
```

El tipo **dim3**, es un tipo predefinido en **CUDA** y representa un vector de 3 componentes de tipo **unsigned int**. Este tipo puede ser usado en el código de la CPU de la siguiente forma:

```
dim3 BT(8, 8, 4); // define e inicializa BT
tx = BT.x;        // acceso al primer elemento de BT (.x)
ty = BT.y;        // acceso al segundo componente de BT (.y)
tz = BT.z;        // acceso al tercer componente de BT (.z)
```

Las variables **blockDim** y **gridDim** se definen en la invocación del kernel (por defecto, estas variables están inicializadas a 1). Las variables **threadIdx** y **blockIdx** se definen en la GPU al lanzar la ejecución de un bloque con sus correspondientes *threads*. Por ejemplo, si tenemos la siguiente invocación:

```
dim3 dimGrid(200, 300, 50);           // #bloques = 200*30*50 = 300000
dim3 dimBlock(8, 16, 4);              // #threads = 8*16*4 = 512
KERNEL<<<dimGrid, dimBlock>>>( );
```

Las variables predefinidas tienen los siguientes rangos y valores:

- `blockDim.x`: 8, `blockDim.y`: 16, `blockDim.z`: 4
- `gridDim.x`: 200, `gridDim.y`: 300, `gridDim.z`: 50
- `threadIdx.x`: [0..7], `threadIdx.y`: [0..15], `threadIdx.z`: [0..3]
- `blockIdx.x`: [0..199], `blockIdx.y`: [0..299], `blockIdx.z`: [0..49]

El número de bloques que se ejecutarán serán 300.000 y en cada uno de estos bloques se lanzarán 512 *threads*. Eso hace que para resolver este KERNEL, la GPU ejecutará 153.600.000 *threads* en total

Un buen conocimiento y correcto uso de estas variables es fundamental para programar eficientemente en **CUDA**.

2.3 Puzzle 1D

El primer código con el que vamos a trabajar es el siguiente:

```
void puzzle1DSeq(int N, float *z, float *x, float *y) {
    int i;
    for (i=0; i<N; i++)
        z[i] = 0.5*x[i] + 0.75*y[i] + x[i]*y[i];
}
```

En primer lugar, hay que implementar la versión paralela en **CUDA**, completando este código:

```
__global__ void puzzle1DPAR(int N, float *z, float *x, float *y) {
    . . .
}
```

En la primera versión a implementar podéis suponer que el tamaño del problema es múltiplo del número de *threads*, y que el número total de *threads* coincide con el tamaño del problema.

Editando el fichero `puzzle1D.cu` que encontraréis en el directorio **Puzzle1D** tenéis que hacer lo siguiente:

1. Completad el código para su ejecución en la GPU. Ejecutadlo con 1024 *threads* y tamaño de problema múltiplo de 1024.
2. Cambiad el tamaño para que NO sea múltiplo del número de *threads*. Modificad el código, dónde sea necesario, para hacer que funcione correctamente.

3. ¿Cómo modificarías la implementación si los datos no caben en la memoria de la GPU?
4. ¿Cómo modificarías la implementación si quisiéramos que el número de *Blocks* y *Threads* fuera fijo, independientemente del tamaño del problema?

2.4 Puzzle 2D

En este apartado vamos a trabajar con un problema similar, pero ahora con matrices:

```
void puzzle2DSeq(int Nfil, int Mcol,
                 float z[][Mcol], float x[][Mcol], float y[][Mcol]) {
    int i, j;
    for (i=0; i<Nfil; i++)
        for (j=0; j<Mcol; j++)
            z[i][j] = 0.5*x[i][j] + 0.75*y[i][j] + x[i][j]*y[i][j];
}
```

El mismo código, pero trabajando con punteros sería el siguiente:

```
void puzzle2DSeq(int Nfil, int Mcol,
                 float *z, float *x, float *y) {
    int i, j, ind;
    for (i=0; i<Nfil; i++)
        for (j=0; j<Mcol; j++) {
            ind = i * Mcol + j;
            z[ind] = 0.5*x[ind] + 0.75*y[ind] + x[ind]*y[ind];
        }
}
```

De este código os vamos a pedir 3 versiones:

1. En la primera versión cada *thread* se va a ocupar de **1 columna** de la matriz resultado.
2. En la segunda versión cada *thread* se va a ocupar de **1 fila** de la matriz resultado.
3. En la última versión cada *thread* se va a ocupar de **1 elemento** de la matriz resultado.

En este caso vamos a implementar directamente el código para que acepte cualquier tamaño de matriz. En particular tendremos matrices NO cuadradas y NO múltiplo del número de *threads*. El programa de test con todo el código necesario para probar las 3 versiones está en el fichero [puzzle2D.cu](#) que encontraréis en el directorio [Puzzle2D](#).

2.5 Puzzle 3D

Finalmente, vamos a trabajar con matrices tridimensionales:


```
void puzzle3DSeq(int Ncar, int Nfil, int Mcol,
                float z[][Nfil][Mcol],
                float x[][Nfil][Mcol],
                float y[][Nfil][Mcol]) {
    int i, j, t;
    for (t=0; t<Ncar; t++)
        for (i=0; i<Nfil; i++)
            for (j=0; j<Mcol; j++)
                z[t][i][j] = 0.5*x[t][i][j] + 0.75*y[t][i][j]
                    + x[t][i][j]*y[t][i][j];
}
```

El mismo código, pero trabajando con punteros sería el siguiente:

```
void puzzle3DSeq(int Ncar, int Nfil, int Mcol,
                float *z, float *x, float *y) {
    int i, j, t, ind;
    for (t=0; t<Ncar; t++)
        for (i=0; i<Nfil; i++)
            for (j=0; j<Mcol; j++) {
                ind = t*Nfil*Mcol + i*Mcol + j;
                z[ind] = 0.5*x[ind] + 0.75*y[ind] + x[ind]*y[ind];
            }
}
```

De este código sólo os vamos a pedir la versión en que **1 thread calcula 1 elemento** de la matriz resultado. Tenéis que implementar el código para que acepte matrices NO cuadradas. El programa de test con todo el código necesario para comprobar el resultado está en el fichero [puzzle3D.cu](#) que encontraréis en el directorio [Puzzle3D](#).

2.6 Profiling

[nvprof](#) es la herramienta que nos proporciona NVIDIA para hacer profiling de nuestras aplicaciones [CUDA](#). Los flags que podéis utilizar con nvprof son los siguientes:

- [--print-gpu-summary](#), genera un sumario de tiempos de ejecución de los kernels y transferencias entre GPU y CPU
- [--print-gpu-trace](#), además de la información anterior genera una traza de los kernels y transferencias entre CPU y CPU

[nvprof](#) es una herramienta muy útil para los programadores de [CUDA](#). En las siguientes sesiones iremos ampliando las cosas que podemos hacer con [nvprof](#), y con otras herramientas.

2.7 Consideraciones Finales

Los ficheros `job.sh` ya incluyen los diferentes tamaños de matriz a probar y la forma de invocar `nvprof`. Como queremos que los tamaños de matriz no sean múltiplo del número de *threads*, lo hemos forzado en el código. Utilizando una sentencia como ésta:

```
if (N % nThreads == 0) N--;
```

Para facilitaros las cosas, y sólo para evitar que os quedéis encallados, hemos incluido un directorio en el que están todos los kernels solucionados.

3 SESIÓN 3 – REDUCCIONES

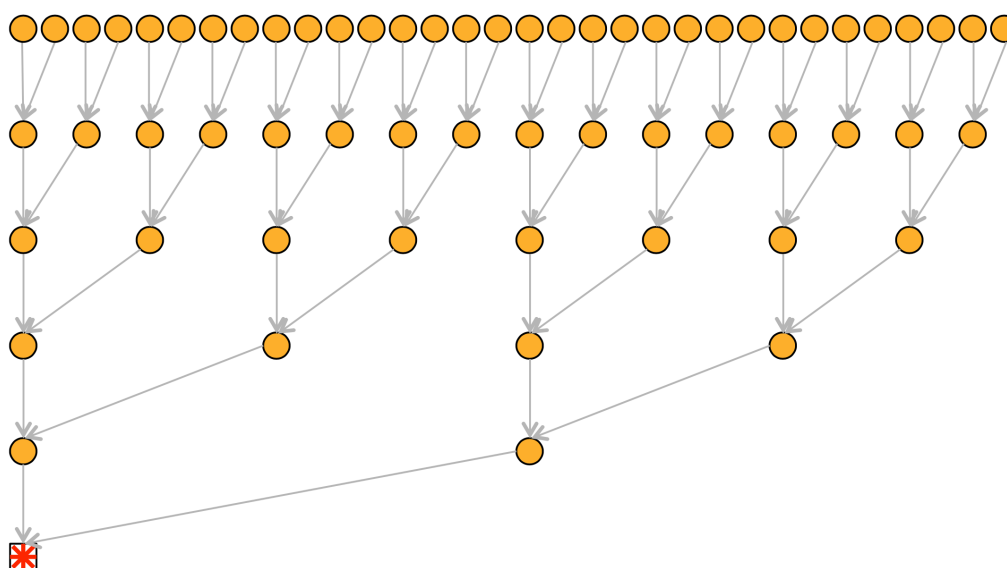
Uno de los algoritmos típicos a resolver en cualquier entorno paralelo es una reducción. En esta sesión vamos a diseñar una reducción paralela en CUDA. Empezaremos con una solución muy simple y la iremos mejorando hasta llegar a un algoritmo bastante eficiente. Las diferentes mejoras nos servirán para ilustrar algunas optimizaciones aplicables a cualquier aplicación CUDA.

3.1 Introducción

En esta sesión vamos a trabajar con un algoritmo muy común, fácil de implementar en **CUDA**, pero difícil de optimizar. Es lo que conocemos como una reducción. Un ejemplo secuencial de reducción es la suma de los elementos de un vector:

```
sum = 0.0;
for (i=0; i<N; i++)
    sum = sum + v[i];
```

El esquema típico para realizar una reducción en paralelo es construir un árbol similar a éste:



Empezamos con 2^n threads, que realizan 2^n sumas parciales en paralelo. En el siguiente paso 2^{n-1} threads dejan de trabajar y el resto realiza 2^{n-1} sumas parciales. Seguimos así, hasta que al final sólo queda un thread que hace la suma final. Este tipo de algoritmo necesita que el número de threads (el número de elementos del vector) a utilizar sea potencia de 2.

Todas las ideas de esta sesión las hemos obtenido de un tutorial de NVIDIA que podéis encontrar en el siguiente enlace: <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>

Por simplicidad, supondremos que el tamaño de vector que utilizaremos en todas las versiones es múltiplo del número de threads ($n\text{Threads} = 512$ y $N = 16.777.216 = 2^{24}$). Es muy fácil modificar estos

algoritmos para que funcionen con cualquier valor de N.

3.2 Kernel 01 (main01.cu)

Este test os lo daremos ya hecho y sólo tenéis compilarlo y ejecutarlo.

Utilizamos un kernel que calcula la suma de los elementos de un vector de 512 elementos. El código es el siguiente:

```
__global__ void Kernel01(float *g_idata, float *g_odata) {
    __shared__ float sdata[512];
    unsigned int s;

    // Cada thread carga 1 elemento desde la memoria global
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    sdata[tid] = g_idata[i];
    __syncthreads();

    // Hacemos la reduccion en la memoria compartida
    for(s=1; s < blockDim.x; s *= 2) {
        if (tid % (2*s) == 0)
            sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    // El thread0 escribe el resultado de su bloque en memoria global
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
```

Cada uno de los *thread blocks* da como resultado una suma parcial que se guarda en `g_odata`. Para compilar el programa sólo hay que hacer:

```
make kernel01.exe
```

Observad, que el compilador nos da información útil de cómo estamos utilizando los recursos hardware: cuántos registros utilizamos por thread, cuánta memoria compartida utilizamos por Thread Block, etc..

Para ejecutarlo hay que enviarlo al sistema de colas, tal y cómo hicimos en la sesión anterior:

```
sbatch job.sh
```

Modificando adecuadamente el fichero `job.sh` podremos ejecutar las diferentes versiones de la reducción.

En este código sólo invocamos el kernel 1 vez, de tal forma que al acabar tenemos un vector de sumas parciales con tantos elementos como `nBlocks`. Para calcular el resultado final hay que hacer la suma final de estos elementos.

Este código se muestra a continuación⁵:

```
// Ejecutar el kernel
Kernel01<<<nBlocks, nThreads>>>(d_v, d_w);
// Obtener el resultado parcial desde el host
cudaMemcpy(h_w, d_w, numBytesW, cudaMemcpyDeviceToHost);
SUM = 0.0;
for (i=0; i<nBlocks; i++)
    SUM = SUM + h_w[i];
```

Para obtener los tiempos de ejecución del código que se ejecutan en la CPU y la GPU es necesario utilizar Events de **CUDA**. Utilizaremos el mismo esquema que mostramos en la Sesión anterior. Para comparar rendimientos de forma más visual, calcularemos el “ancho de banda” de sumar los elementos del vector (tamaño del vector a sumar en bytes/tiempo de ejecución).

La ejecución de este código dará algo parecido a esto:

```
KERNEL 01
Vector Size: 16777216
nThreads: 512
nBlocks: 32768
Tiempo Total 7.464704 milseg
Ancho de Banda 8.990 GB/s
TEST PASS
```

Estos tiempos corresponden a la ejecución en una GPU diferente a la utilizada en el laboratorio de TGA.

Al final del programa se comprueba que el resultado es correcto. En caso contrario aparecería un mensaje como éste:

```
...
TEST FAIL
```

3.3 Kernel 02 (main02.cu)

Por ejemplo, si $N=16 \cdot 1024 \cdot 1024$, en el Kernel01 lanzaríamos 32.768 *thread blocks*, eso nos obligaría a sumar un vector de 32.768 elementos en la CPU. Una optimización que podría hacerse es lanzar una nueva invocación del mismo kernel con $(32.768/512) = 64$ *thread blocks* y 512 *threads*. Así sólo necesitaremos sumar un vector de 64 elementos en la CPU. Además, nos aprovecharemos de que el vector de 32.768 elementos ya está almacenado en la GPU.

Esta optimización tenéis que hacerla vosotros.

Para los diferentes kernels se recomienda generar un nuevo fichero. En este caso con el nombre **main02.cu**. De esa forma podréis usar el **Makefile** para compilarlos con:

⁵Este es el código que vamos a evaluar. Para comparar las diferentes versiones calcularemos el tiempo de ejecución de esta porción de código.

```
make kernel02.exe
```

3.4 Kernel 03 (main03.cu)

En el kernel que hemos implementado hay dos problemas. El más importante es que la ejecución de los *warps* es muy ineficiente, en muy pocas iteraciones estamos utilizando muy pocos *threads* por warp, y además de forma dispersa. El segundo problema es que utilizamos una operación que es muy costosa: Teniendo en cuenta que es una división por un valor potencia de 2, podríamos utilizar máscaras de bits y operaciones lógicas. Para resolver los 2 problemas y utilizando como punto de partida el código del [main02.cu](#), simplemente debéis sustituir esta porción de código:

```
// Hacemos la reduccion en la memoria compartida
for(s=1; s < blockDim.x; s *= 2) {
    if (tid % (2*s) == 0)
        sdata[tid] += sdata[tid + s];
    __syncthreads();
}
```

Por esta otra:

```
// Hacemos la reduccion en la memoria compartida
for(s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;
    if (index < blockDim.x)
        sdata[index] += sdata[index + s];
    __syncthreads();
}
```

Este código mejora el uso de los *threads*, siguen siendo pocos, pero son consecutivos, y además evita el uso de operaciones costosas (%).

3.5 Kernel 04 (main04.cu)

Si analizamos el patrón de acceso a los datos, veremos que el patrón es muy desafortunado, utilizando strides potencia de dos (2, 4, 8, ...) lo que provoca numerosos conflictos al acceder a los bancos de memoria. Necesitamos modificar el patrón de acceso para que los datos sean accedidos en posiciones consecutivas de memoria.

Además, el kernel que hemos implementado utiliza los *threads* de forma muy ineficiente. Primero trabajan todos los *threads*, luego la mitad, luego la cuarta parte, ... Teniendo en cuenta que **CUDA** ejecuta los *threads* en grupos de 32 (del 0 al 31, del 32 al 63, ...), eso provoca que el rendimiento sea muy deficiente. A partir de la tercera iteración en cada *warp* sólo se ejecutan unos pocos *threads*.

Para solventar estos problemas, simplemente debéis sustituir esta porción de código:

```
// Hacemos la reduccion en la memoria compartida
for(s=1; s < blockDim.x; s *= 2) {
    int index = 2 * s * tid;
    if (index < blockDim.x)
        sdata[index] += sdata[index + s];
    __syncthreads();
}
```

Por esta otra:

```
// Hacemos la reduccion en la memoria compartida
for (s=blockDim.x/2; s>0; s>>=1) {
    if (tid < s)
        sdata[tid] += sdata[tid + s];
    __syncthreads();
}
```

Podéis comprobar fácilmente que estamos accediendo a posiciones consecutivas de memoria y que el uso de los *threads* es más eficiente.

3.6 Kernel 05 (main05.cu)

Si estudiamos detenidamente el último kernel podemos observar que todos los *threads* traen 1 dato desde la memoria global a la compartida, pero que, en la primera iteración del bucle, la mitad de los *threads* ya no tienen nada que hacer. Esto supone una pérdida de recursos muy importante que podemos resolver fácilmente.

La solución es sencilla:

- dividir a la mitad el número de *thread blocks* que ejecutamos,
- sustituir la parte inicial del kernel, de tal forma que cada *thread* lea dos datos de la memoria global, los sume, y guarde el resultado en la memoria compartida.
- Como resultado, si antes en un *thread blocks* procesábamos M ($2^9 = 512$) datos, ahora procesaremos $2M$ ($2^{10} = 1024$) datos.

Resumiendo, debéis sustituir esta porción de código:

```
// Cada thread carga 1 elemento desde la memoria global
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
sdata[tid] = g_idata[i];
```

Por esta:

```
// Cada thread carga 2 elementos desde la memoria global,
// los suma y los deja en la memoria compartida
unsigned int tid = threadIdx.x;
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];
__syncthreads();
```

No os olvidéis de dividir a la mitad el número de *thread blocks* que ejecutamos.

La mejora de rendimiento que se obtiene es importante.

3.7 Kernel 06 (main06.cu)

Si analizamos los resultados, veremos que el ancho de banda que obtenemos aún está lejos del ancho de banda de la memoria global de la GPU. Hemos de hacer optimizaciones más agresivas.

El número de *threads* activos se va reduciendo en cada iteración del bucle. Llegará un punto en que sólo tendremos 32 *threads* activos (1 Warp). Dentro de 1 *warp* todas las instrucciones se ejecutan de forma síncrona [se ejecutan los 32 *threads* simultáneamente]. Eso implica que la instrucción `__syncthreads()` no es necesaria cuando se ejecuta el último Warp, y que podemos desenrollar la parte final del bucle.

Resumiendo, debéis sustituir esta porción de código:

```
// Hacemos la reduccion en la memoria compartida
for (s=blockDim.x/2; s>0; s>>=1) {
    if (tid < s)
        sdata[tid] += sdata[tid + s];
    __syncthreads();
}
```

Por esta otra:

```
// Hacemos la reduccion en la memoria compartida
for (s=blockDim.x/2; s>32; s>>=1) {
    if (tid < s)
        sdata[tid] += sdata[tid + s];
    __syncthreads();
}

// desenrollamos el ultimo warp activo
if (tid < 32) {
    volatile double *smem = sdata;
    smem[tid] += smem[tid + 32];
    smem[tid] += smem[tid + 16];
    smem[tid] += smem[tid + 8];
    smem[tid] += smem[tid + 4];
    smem[tid] += smem[tid + 2];
    smem[tid] += smem[tid + 1];
}
```


De este código que ejecutamos en el último *warp* activo hay que decir algunas cosas:

- La parte final no necesita sincronización explícita. Sólo se está ejecutando 1 *warp* y todos los *threads* del *warp* están ejecutando la misma instrucción simultáneamente.
- Los 32 *threads* ejecutan todo el código. Algunos de ellos hacen operaciones inútiles, e incluso, sin sentido. Pero al acabar, en el *thread* 0 tenemos el resultado que estamos buscando.
- La sentencia `volatile double *sem = sdata` es un recurso del lenguaje que informa al compilador que esa variable no se puede guardar en cache. Podéis comentar la sentencia y veréis que el programa deja de funcionar.
- Si utilizamos C++, podemos escribir un código con templates, completamente desenrollado, que funcione para cualquier tamaño de Thread Block. Eso nos obligaría a escribir el código de todos los posibles casos. En la página web de NVIDIA se puede encontrar este código.

3.8 Kernel 07 (main07.cu) [ÚLTIMO TEST]

La última optimización mezcla parte secuencial [en los *thread blocks*] con el código que ya tenemos. La idea es sencilla:

- Reducir el número de *thread blocks* , por ejemplo a 2048, con 512 *threads* en cada uno.
- El primer paso del algoritmo es sumar, secuencialmente, en cada *thread* los $N/(2048 \cdot 512)$ datos que le corresponden y dejar la suma parcial en la memoria compartida.
- A continuación, cada *thread blocks* calcula la suma parcial, utilizando el mismo kernel de la versión anterior.
- En el host, es suficiente con hacer una invocación del kernel, y dejar a la CPU la suma final de las 2048 sumas parciales.

Resumiendo, debéis sustituir esta porción de código:

```
// Cada thread carga 2 elementos desde la memoria global,  
// los suma y los deja en la memoria compartida  
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;  
sdata[tid] = g_idata[i] + g_idata[i+blockDim.x];  
__syncthreads();
```

Por esta otra:

```
// Cada thread realiza la suma parcial de los datos que le  
// corresponden y la deja en la memoria compartida  
unsigned int tid = threadIdx.x;  
unsigned int i = blockIdx.x*(blockDim.x*2) + threadIdx.x;
```

```
unsigned int gridSize = blockDim.x*2*gridDim.x;
sdata[tid] = 0;
while (i < N) {
    sdata[tid] += g_idata[i] + g_idata[i+blockDim.x];
    i += gridSize;
}
__syncthreads();
```

Para que todo funcione, hay que incluir la N como parámetro del kernel y modificar el código del host adecuadamente.

3.9 Profiling

En las antiguas versiones de **CUDA** utilizábamos para hacer profiling la herramienta **nvprof**. En la actualidad, **nvprof** sigue existiendo, pero la información que ofrece es muy limitada.

La nueva herramienta de profiling que os vamos a mostrar es **ncu**. Hemos montado un fichero llamado **jobInfoNCU.sh**, que podéis lanzar a ejecución con un **sbatch** para que os muestre las diversas opciones y el help de **ncu**.

Por ahora, usaremos estas tres posibilidades (podéis utilizar el fichero **jobNCU.sh**):

```
ncu                ./kernel01.exe
ncu --set full      ./kernel01.exe
ncu --set detailed  ./kernel01.exe
```

Lo que muestra cada una de estas llamadas es la información de profiling de un conjunto de secciones predefinidas. Si queréis saber que secciones pertenecen a cada conjunto podéis hacer:

```
ncu --list-sets
```

Esta herramienta muestra mucha información, os recomendamos que la leáis detenidamente.

3.10 Consideraciones Finales

En esta sesión os damos como punto de entrada el fichero **main01.cu**. A partir de aquí hay que ir construyendo los diferentes kernels con las ideas que os hemos dado.

Para facilitaros las cosas, y sólo para evitar que os quedéis encallados, hemos incluido un directorio en el que están todos los kernels solucionados.

4 SESIÓN 4 – PRODUCTO DE MATRICES

El producto de matrices es uno de los primeros algoritmos a paralelizar en cualquier entorno paralelo. Es un algoritmo sencillo de paralelizar, pero sobre todo es uno de los algoritmos más utilizados en las aplicaciones numéricas.

Otro de los puntos importantes de la sesión será ilustrar el uso de la memoria compartida y los beneficios que da.

4.1 Introducción

En esta sesión vamos a trabajar con un algoritmo muy conocido, el producto de matrices. Un posible algoritmo secuencial podría ser el siguiente:

```
for (i=0; i<N; i++)  
  for (j=0; j<M; j++)  
    for (k=0; k<P; k++)  
      C[i][j] = C[i][j] + A[i][k]*B[k][j];
```

4.2 Kernel 00 (MM00.cu)

Este test os lo damos ya hecho, sólo tenéis que compilarlo y ejecutarlo.

Para compilar el programa sólo hay que hacer:

```
make kernel00.exe
```

Para ejecutarlo hay que enviarlo al sistema de colas, tal y cómo hicimos en las sesiones anteriores.

Al ejecutar el kernel podemos pasarle 2 parámetros. El primero es el tamaño del problema (este kernel sólo funciona para matrices cuadradas y múltiplo del número de *threads* en cada dimensión). El segundo es un flag ('Y' o 'N') para indicar si queremos comprobar el resultado. Una ejecución podría ser la siguiente:

```
./kernel00.exe 640 Y
```

Ejecuta el kernel con matrices de 640×640 elementos y comprueba el resultado⁶. El resultado de esta ejecución podría ser el siguiente (todos los resultados que aparecen en esta sesión corresponden a la ejecución con una tarjeta Tesla K40c):

⁶El código que os proporcionamos permite probar la memoria PINNED. Simplemente hay que cambiar el flag correspondiente en el [Makefile](#).

```

KERNEL 00
Dimensiones: 640x640
nThreads: 32x32 (1024)
nBlocks: 20x20 (400)
NO usa Pinned Memory
Tiempo Global: 7.702144 milseg
Tiempo Kernel: 4.764320 milseg
Rendimiento Global: 68.07 GFLOPS
Rendimiento Kernel: 110.04 GFLOPS
TEST PASS

```

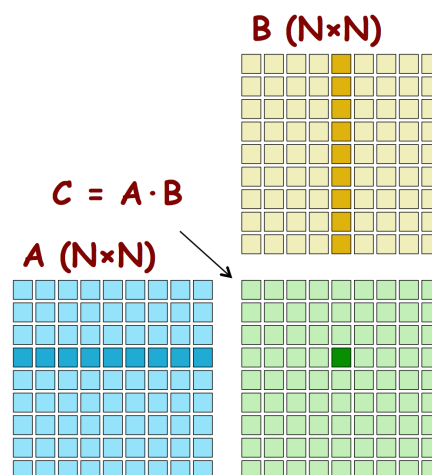
El programa calcula el rendimiento del kernel incluyendo el movimiento de datos entre CPU y GPU (usa **Tiempo Global**), y el rendimiento del kernel exclusivamente (usa **Tiempo Kernel**).

Este kernel es el que vimos en clase. Cada *thread* calcula un elemento de la matriz resultado (**C[i][j]**). Cada *thread* ha de leer una fila de la matriz A (**A[i][...]**) y una columna de B (**B[...][j]**). El código del kernel, y una figura ilustrativa, se muestran a continuación:

```

__global__ void Kernel00(int N, float *A, float *B, float *C) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float tmp = 0.0;
    for (int k=0; k<N; k++)
        tmp += A[row*N+k] * B[k*N+col];
    C[row*N+col] = tmp;
}

```



Se puede comprobar que este kernel funciona perfectamente cuando el tamaño del problema es múltiplo del número de *threads* (**32, en nuestro ejemplo**) en cada dimensión. Pero sólo funciona si N es múltiplo del número de *threads*, si esa condición no se cumple, el código será erróneo.

Por ejemplo, si ejecutamos el programa con matrices de 641×641 elementos:

```
./kernel00.exe 641 Y
```

El resultado que obtenemos es el siguiente:

```
KERNEL 00
Dimensiones: 641x641
nThreads: 32x32 (1024)
nBlocks: 20x20 (400)
NO usa Pinned Memory
Tiempo Global: 8.915040 milseg
Tiempo Kernel: 5.976320 milseg
Rendimiento Global: 59.09 GFLOPS
Rendimiento Kernel: 88.14 GFLOPS
0:640: 167.605087 - 0.000000 = 167.605087
TEST FAIL
```

Se puede ver que el número de *thread blocks* no es correcto, el elemento (0, 640) de la matriz resultado no se está calculando. Es fácil comprobar que el valor de *nBlocks* es incorrecto. Debería ser 21×21 . Pero, si modificamos el código para subsanar este error, el resultado seguiría siendo incorrecto. Algunos *threads* acceden a posiciones de memoria que no existen.

4.3 Kernel 01 (MM01.cu)

El kernel 00 sólo funciona cuando las dimensiones del problema son múltiplo del número de *threads* en cada dimensión. Modificad el kernel para que funcione con matrices de cualquier dimensión y con matrices no cuadradas.

Hay que modificar bastantes cosas:

- El `main`, para leer de la línea de comandos las 3 dimensiones del problema:

$$C(N \times M) \leftarrow A(N \times P) \cdot B(P \times M)$$

Si queremos que el problema a resolver tenga las siguientes dimensiones: $N = 639$, $M = 641$ y $P = 1023$. La invocación del kernel debería ser algo parecido a esto:

```
./kernel01.exe 639 641 1023 Y
```

- La creación e inicialización de las matrices y las transferencias entre CPU y GPU.
- El kernel, para que tenga en cuenta que el tamaño del problema no es múltiplo del número de *threads*. Los *thread blocks* de los bordes han de hacer menos cálculos y evitar que escriban en memoria.
- La invocación del kernel, para que lance el número correcto de *thread blocks*.
- El cálculo de los GFLOPS.
- No os olvidéis de revisar los parámetros de la invocación de la función `TestMM`.

Si analizamos el kernel que hemos utilizado en este programa tenemos que:

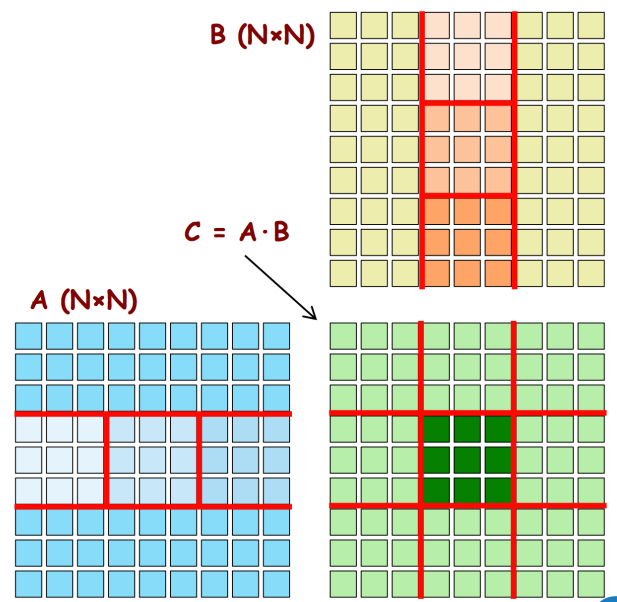
- Cada *thread* calcula un elemento de la matriz C.
 - Cada fila de A es leída N veces desde la memoria global.
 - Cada columna de B es leída N veces desde la memoria global.
- Se desperdicia mucho ancho de banda.
- Tenemos un mal equilibrio entre cálculo y ancho de banda. En definitiva, es un código mejorable.

4.4 Kernel 10 (MM10.cu)

En este kernel vais a implementar el producto de matrices a bloques que vimos en las clases de teoría. Os incluimos las dos transparencias dónde se explicaba el kernel.

$C = A \cdot B$ (tamaño $N \times N$)

- Cada block thread calcula un bloque de datos de $M \times M$ elementos de la matriz C.
- El algoritmo itera N/veces. En cada iteración
 - Traemos un bloque $M \times M$ de A a la memoria compartida. La matriz A se lee N/M veces.
 - Traemos un bloque $M \times M$ de B a la memoria compartida. La matriz B se lee N/M veces.
- El valor de M es importante:
 - Las submatrices han de caber en la memoria compartida.
 - Necesitamos $M \times M$ threads por bloque.
- Utilizamos menos ancho de banda. El equilibrio entre cálculo y ancho de banda mejorará.



```
__global__ void MMkernel2(float *dA, float *dB, float *dC, int N) {
    __shared__ float sA[M][M]; __shared__ float sB[M][M];
    int thx = threadIdx.x; int thy = threadIdx.y;
    int row = blockIdx.y * M + thy;
    int col = blockIdx.x * M + thx;
    float tmp = 0.0;
    for (s=0; s<(N/M); s++) {
        sA[thy][thx] = dA[row*N + s*M + thx];
        sB[thy][thx] = dB[(s*M + thy)*N + col];
        __syncthreads();
        for (int k=0; k<M; k++)
            tmp += sA[thy][k] * sB[k][thx];
        __syncthreads();
    }
    dC[row*N+col] = tmp;
}
```

Submatrices en la Memoria Compartida

Todos los threads del bloque cooperan para cargar las submatrices A y B en la memoria compartida

Antes de empezar a calcular hay que asegurarse que todos los threads hayan acabado de cargar las submatrices A y B.

Antes de continuar hay que asegurarse que todos los threads hayan acabado los cálculos que le corresponden.

Implementad esta nueva versión del kernel. Utilizad como punto de partida el fichero [MM01.cu](#). Tenéis que implementar el kernel pensando en que ha de aceptar matrices NO cuadradas:

$$C(N \times M) \leftarrow A(N \times P) \cdot B(P \times M)$$

En principio, en esta versión sólo hay que preocuparse de que funcione correctamente con dimensiones de matriz múltiplo del número de *threads*. Por ejemplo, así:

```
./kernel10.exe 640 512 1024 Y
```

4.5 Kernel 11 (MM11.cu)

Modificad el kernel para que funcione con matrices de cualquier dimensión. Si utilizáis como punto de partida el código de [MM10.cu](#), sólo hay que modificar el kernel. Queremos que el nuevo kernel funcione con algo parecido a esto:

```
./kernel10.exe 639 641 1023 Y
```

¡Atención con esta versión! El código no es trivial. Es un algoritmo bastante complejo. Procurad que no se pierda mucho rendimiento con respecto a la versión del kernel10.

4.6 Pinned Memory

Se puede modificar el [Makefile](#) de la siguiente forma:

```
PROG_FLAGS = -DPINNED=1
```

Al cambiar este flag el programa utilizará Pinned Memory (obtenida con [cudaMallocHost](#)) en vez de memoria paginada (obtenida con [malloc](#)).

Compilad y ejecutad de nuevo los programas anteriores y observad como cambia el rendimiento de nuestra aplicación.

4.7 Otras cosas a probar

Algunas pruebas interesantes a realizar son las siguientes:

- Probad diferentes dimensiones de Block Thread: 16×16 , 8×8 , 16×32 , 8×16 , etc.. Algunas de ellas (las no cuadradas) requieren que modifiquéis ligeramente el código.

```
PROG_FLAGS = -DSIZE=16
```

- Modificad `MM01.cu` y `MM11.cu` para que se ejecute en un bucle diferentes invocaciones del Kernel01 y Kernel11. El objetivo es comprobar cómo cambia el rendimiento al aumentar el tamaño, y la pérdida de rendimiento cuando el tamaño de las matrices no es múltiplo del número de *threads*. El código para ver la influencia del tamaño podría ser algo así (usamos matrices cuadradas con dimensiones múltiplo del número de *threads*):

```
for (N=128; N<2048; N=N+32)
    Kernel01<<<dimGrid, dimBlock>>>(N, N, N, d_A, d_B, d_C);
```

El código para ver cómo influye que el tamaño del problema no sea múltiplo del número de *threads* podría ser algo así (usamos matrices cuadradas, pero nos aseguramos que la mayoría de los tamaños utilizados no sean múltiplo del número de *threads*):

```
for (N=1024; N<1256; N=N+2)
    Kernel01<<<dimGrid, dimBlock>>>(N, N, N, d_A, d_B, d_C);
```

Obviamente, habrá que modificar el código adecuadamente para obtener los tiempos de ejecución. En este caso, no será necesario comprobar el resultado. Ya sabemos que los kernels funcionan bien.

4.8 Un desafío

Si miramos el rendimiento de pico de la NVIDIA RTX 3080 vemos que es de 29,77 TFLOPs. Con el algoritmo que utiliza la memoria compartida, podemos llegar (más o menos) a los 2,4 TFLOPs. Estamos muy lejos. ¿Se os ocurre alguna estrategia, que no sea usar *Tensor Cores*, para mejorar estos rendimientos? A lo mejor, ésto os da alguna idea: "https://github.com/NVIDIA/cutlass/blob/main/media/docs/programming_guidelines.md".

4.9 Consideraciones Finales

Igual que hicimos en la sesión anterior, para facilitaros las cosas, y sólo para evitar que os quedéis encallados, hemos incluido un directorio en el que están todos los kernels solucionados.

5 SESIÓN 5 – MULTI GPU

Nuestro servidor tiene varias GPUs. El objetivo de la Sesión es obvio: utilizar en una misma aplicación las GPUs disponibles.

Un tema lateral que aparece en este caso es cómo tomar tiempos de ejecución cuando tenemos varias GPUs ejecutando en paralelo una misma aplicación.

5.1 Introducción

En esta sesión vamos a trabajar con varias GPUs. Vamos a presentar las herramientas que ofrece **CUDA** para utilizar en una misma aplicación las 4 RTX 3080 que tenemos en el servidor.

Para trabajar con varias GPUs hay que descomponer el problema en partes (similar a lo que hay que hacer para trabajar con streams) y distribuirlo entre las diversas GPUs. En general, será necesario disponer de algún tipo de mecanismo para que las diversas GPUs se comuniquen. Tanto para sincronizarse, como para compartir datos.

Las razones para trabajar con varias GPUs son siempre las mismas:

- Aumentar la velocidad de cálculo
- El tamaño del problema supera la capacidad de memoria de 1 GPU

Existen diversos escenarios posibles:

- Un único nodo con varias GPUs
- Varios nodos conectados en red con 1 o varias GPUs por nodo

Por motivos obvios, nos centraremos en el primer caso.

Por definición, todas las llamadas **CUDA** se ejecutan en la current GPU. Para trabajar con varias GPUs sólo necesitamos una rutina que nos permita cambiar la current GPU. Esa rutina es:

```
cudaError_t cudaSetDevice(int device)
```

El parámetro device es simplemente un número entero entre 0 y N-1, siendo N el número de GPUs disponibles. Para saber el número de GPUs disponibles hay que invocar esta función:

```
cudaError_t cudaGetDeviceCount(int *count)
```

Al acabar la rutina, `*count` tendrá el número de GPUs disponibles en el sistema. Esta rutina ya la utilizamos en la Sesión01 (podéis mirar el código), en la aplicación **ScanDevices**.

El siguiente código permite ejecutar varios kernels en las GPUs disponibles:

```

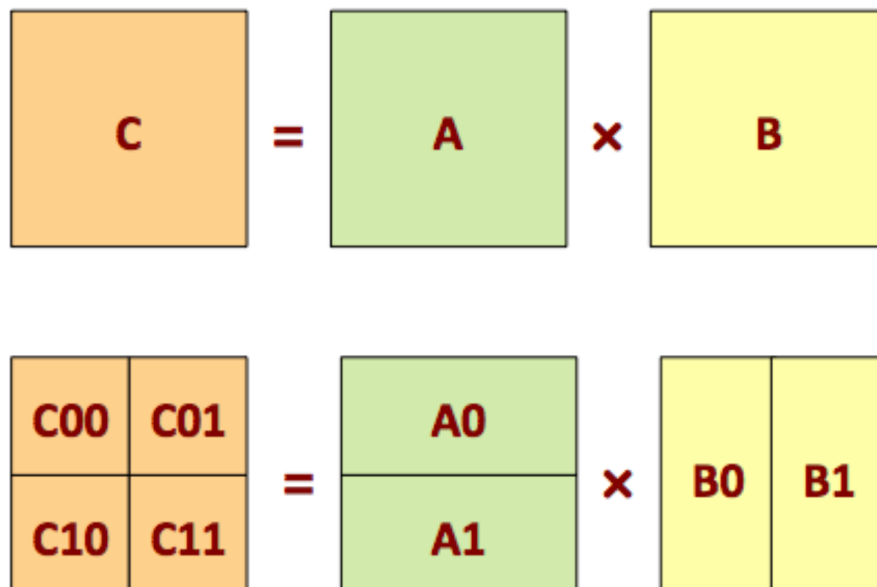
cudaGetDeviceCount(&count);
for (dev = 0; dev < count; dev++) {
    . . .
    cudaSetDevice(dev);
    kernel<<< . . . >>>(. . .);
    . . .
}

```

5.2 KERNEL MultiGPU

Como ejemplo vamos a utilizar el producto de matrices que ya vimos en la sesión anterior y para simplificar el código vamos a realizar un algoritmo en el que no tengamos que mover datos entre GPUs.

La idea básica del algoritmo que vamos a implementar se muestra en la siguiente figura:



Para simplificar las cosas supondremos que las matrices son cuadradas, y mejor aún, que las dimensiones son potencia de 2 (p.e.: 2048×2048). La distribución de cálculos es la siguiente:

- GPU 0: calculamos $C_{00} = A_0 \cdot B_0$, necesita A_0 y B_0
- GPU 1: calculamos $C_{01} = A_0 \cdot B_1$, necesita A_0 y B_1
- GPU 2: calculamos $C_{10} = A_1 \cdot B_0$, necesita A_1 y B_0
- GPU 3: calculamos $C_{11} = A_1 \cdot B_1$, necesita A_1 y B_1

Para hacer el cálculo de producto de matrices, podéis utilizar el kernel a bloques de la sesión anterior. En concreto el que funcionaba para tamaños de problema múltiplo del número de *threads*.

En los ficheros de la sesión encontraréis un fichero ([MM4GPUs.cu](#)). En este fichero, está parte del algoritmo implementado. En concreto encontraréis el cálculo de C00, así como la infraestructura necesaria para crear las matrices, comprobar el resultado y tomar tiempos.

En una aplicación real, recibiríamos las matrices completas (A, B y C), y por programa, deberíamos construir las submatrices que necesitamos. Teniendo en cuenta que el objetivo de la práctica es usar varias GPUs, y para simplificar el trabajo, hemos creado las matrices A0, A1, B0, B1, C00, C01, C10 y C11 por separado.

Para compilar y ejecutar la aplicación utilizaremos las mismas herramientas de otras sesiones:

```
make kernel4GPUs.exe
sbatch job.sh
```

La primera tarea de la Sesión consiste en completar el programa para que calcule el producto de la matriz C completa. Al ejecutar la aplicación, en el caso de que queráis comprobar el resultado, es conveniente que no utilicéis tamaños de matriz muy grandes (p.e. 1024×1024).

5.3 Tomando Tiempos

Cuando queremos averiguar el tiempo de ejecución de una aplicación **CUDA** que se ejecuta en una sola GPU, utilizábamos eventos de la siguiente forma:

```
float TiempoTotal;
cudaEvent_t E0, E3;
cudaEventCreate(&E0); cudaEventCreate(&E3);
cudaEventRecord(E0, 0);
//AQUI ESTARIA LA PORCION DE CODIGO A EVALUAR
cudaEventRecord(E3, 0); cudaEventSynchronize(E3);

cudaEventElapsedTime(&TiempoTotal, E0, E3);
// En TiempoTotal tenemos el tiempo de ejecucion del codigo en ms
cudaEventDestroy(E0); cudaEventDestroy(E3);
```

Lo que hacemos es registrar eventos (E0 y E3) en la cola de ejecución de la GPU y sincronizarnos con su finalización. Junto con el evento el sistema guarda el instante de tiempo en el que acaba. Los eventos se ejecutan cuando todas las llamadas previas a **CUDA**, en esa GPU y en ese stream, han terminado. Para acabar sólo necesitamos la diferencia de tiempos entre los eventos E0 y E3.

Cuando estamos trabajando con varias GPUs tenemos que hacer alguna cosa más. Para empezar, sólo podemos registrar eventos en nuestra propia GPU y no podemos acceder a la información de un evento en otra GPU. Pero, lo que sí podemos hacer, es sincronizarnos con un evento de otra GPU.

La idea es la siguiente. En la GPU 0, calculamos el tiempo de ejecución global, guardando un evento al principio del cálculo (E0) y otro al final (E3). Pero para que la toma de tiempos sea correcta nos tenemos que asegurar que la ejecución en las otras GPUs haya terminado. Lo que hacemos es registrar un evento al final del cálculo en cada GPU: X1, X2 y X3. Y en la GPU 0 nos sincronizaremos con los 3 eventos.

El código necesario para tomar tiempos ya está en la aplicación que os hemos pasado.

En otras sesiones, también calculábamos el tiempo de ejecución del kernel. En este caso no es viable, porque la sincronización necesaria influye en el tiempo global de la aplicación. Si queremos saber el tiempo de los kernels hay que consultar los datos que nos da `nvprof`.

También es posible calcular el tiempo que tarda la aplicación utilizando `nvprof`. En concreto, con la siguiente llamada:

```
nsys nvprof --print-gpu-trace ./kernel4GPUs.exe 2048 N
```

En la información que proporciona `nvprof`, podemos encontrar en qué instante de tiempo se inicia cada llamada `CUDA` a evaluar (transferencias y kernels) y cuánto dura su ejecución. Con un poco de esfuerzo es posible calcular el tiempo de ejecución de la aplicación paralela en las 4 GPUs.

5.4 `cudaMemcpyAsync`

Si miráis de nuevo el código, veréis que todas las comunicaciones entre el Host y el Device son del tipo `cudaMemcpyAsync`. En este punto, deberíamos preguntarnos, ¿porqué usamos `cudaMemcpyAsync` y no `cudaMemcpy`? Para responder a esta pregunta, os recomendamos hacer lo siguiente:

1. Ejecutad el código original con la siguiente llamada:

```
nsys nvprof --print-gpu-trace ./kernel4GPUs.exe 2048 N
```

Observad en qué orden se ejecutan las transferencias y los kernels.

2. Modificad el código para que todas las transferencias sean del tipo `cudaMemcpy` y repetid la llamada anterior. Observad de nuevo en qué orden se ejecutan las transferencias y los kernels, y veréis claramente porqué usamos `cudaMemcpyAsync` y no `cudaMemcpy`.

5.5 `CUDA Samples y Documentación`

Si tenéis tiempo es un buen momento para mirar los ejemplos de `CUDA` que publica NVIDIA. Antes, estos ejemplos venían con el software de `CUDA`. Ahora, estos ejemplos actualizados, se pueden obtener en: <https://github.com/NVIDIA/cuda-samples>

El ejercicio que os proponemos es que escojáis uno de los ejemplos (los que hay en el directorio `0_Simple`, son los más asequibles), lo compiléis y lo ejecutéis. Para compilarlo tendréis que modificar el `Makefile` y definir un nuevo `job.sh`.

Y aprovechando, podéis darle un vistazo a la documentación que ofrece `CUDA`. La podréis encontrar en <https://docs.nvidia.com/cuda> (ahí está la información para todas las versiones del compilador).

5.6 Consideraciones Finales

Igual que hicimos en sesiones anteriores, para facilitaros las cosas, y sólo para evitar que os quedéis encallados, hemos incluido un directorio en el que está el kernel solucionado.

6 SESIÓN 6 – STREAMS

Unos de los problemas de las GPUs actuales es la conexión PCIe. Es el bus de comunicación entre la CPU y la GPU, y por ahora uno de los cuellos de botella de la mayoría de las aplicaciones CUDA. Para paliar este problema usaremos streams. El objetivo de la sesión es mostrar el mecanismo definido por NVIDIA para solapar el cálculo con las comunicaciones por el bus PCIe.

6.1 Kernel 00 (stream00.cu)

Para trabajar con streams vamos a utilizar un código muy simple que trabaja con vectores. El código de partida os lo damos ya hecho, sólo tenéis que compilarlo y ejecutarlo.

Para compilar el programa sólo hay que hacer:

```
make kernel00.exe
```

Para ejecutarlo hay que enviarlo al sistema de colas, tal y cómo hicimos en sesiones anteriores.

```
sbatch job.sh
```

Al ejecutar el kernel podemos pasarle 2 parámetros. El primero es el tamaño del problema (en K elementos). El segundo es un flag ('Y' o 'N') para indicar si queremos comprobar el resultado. Una ejecución podría ser la siguiente:

```
./kernel00.exe 10000 Y
```

Ejecuta el kernel00 con vectores de 10000·1024 elementos y comprueba el resultado. El resultado de esta ejecución podría ser el siguiente ⁷:

```
KERNEL 00: NO - Streams
Dimension Problema: 10240000
Invocacion Kernel <<<nBlocks,nKernels>>>(N): <<<10000,1024>>>(N)
nKernels: 1
Usando Pinned Memory
Tiempo Global (00): 8.245632 milseg
Rendimiento Global (00): 500.47 GFLOPS
Tiempo Kernel (00): 2.117824 milseg
Tiempo HtoD (00): 4.566848 milseg
Tiempo DtoH (00): 1.560960 milseg
TEST PASS
```

⁷ Los tiempos que aparecen aquí son orientativos, obtenidos con otra versión del compilador y no tienen por que coincidir con los que obtengáis vosotros.

El programa a evaluar es muy simple:

- Se crean 2 vectores en la CPU y se transfieren a la GPU.
- Se realiza un cálculo con esos vectores para generar un vector resultado.
- Se transfiere de la GPU a la CPU el vector resultado.

El código del kernel se muestra a continuación:

```
--global__ void Kernel0(int N, float a, float b,
                        float *A, float *B, float *C){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j;
    if (i<N) {
        C[i] = a*A[i] + b*B[i];
        for (j=0; j<DUMMY; j++)
            C[i] += a*A[i] + b*B[i];
    }
}
```

Este código no realiza ninguna tarea útil. Simplemente, variando la constante `DUMMY`, podemos equilibrar el coste del cálculo (en tiempo) con el coste de las comunicaciones. La variable `DUMMY` se puede modificar en el `Makefile` antes de compilar. En concreto para `DUMMY == 100` en los resultados de la ejecución anterior, tenemos:

```
...
Tiempo Kernel (00): 2.117824 milseg
Tiempo HtoD (00): 4.566848 milseg
Tiempo DtoH (00): 1.560960 milseg
...
```

Puede verse que el coste de las transferencias entre CPU y GPU es "similar" al coste del kernel.

Podéis lanzar varias ejecuciones variando el valor de `DUMMY` para ver el efecto en el resultado final. Os recomendamos que lanzéis los kernels varias veces. Los tiempos obtenidos no siempre son coherentes.

6.2 Hablemos de Streams

En esta sesión vamos a trabajar con streams. En las tarjetas gráficas de NVIDIA existen 3 colas de ejecución: 1 cola para cálculo y 2 colas para mover datos entre host y GPU, una en cada sentido. Las 3 colas pueden funcionar de forma concurrente, siempre y cuando pertenezcan a streams diferentes.

La definición e inicialización de un stream se muestra a continuación:

```
cudaStream_t stream1;
cudaStreamCreate(&stream1) ;
```

Para soportar la ejecución concurrente es necesario que los datos del host estén almacenados en Pinned Memory:

```
cudaMallocHost(&h1, size);
```

y que las transferencias de información sean asíncronas:

```
cudaMemcpyAsync(d1, h1, size, cudaMemcpyHostToDevice, stream1);
```

Se ve claramente la forma de indicar el **stream** utilizado.

La forma en que se indica en qué stream se ejecuta un kernel es la siguiente:

```
kernel2<<<grid, block, 0, stream1>>>();
```

Finalmente, al acabar la aplicación es necesario destruir el stream:

```
cudaStreamDestroy(stream1);
```

6.3 Kernel 01 (stream01.cu)

Modificad la aplicación anterior para que trabaje con streams.

Si miramos el detalle del código se pueden identificar 4 tareas:

- Copiar A: CPU → GPU
- Copiar B: CPU → GPU
- Kernel cálculo $C \leftarrow \text{Kernel}(a, b, A, B)$
- Copiar C: GPU → CPU

Todas estas tareas trabajan con vectores de N elementos.

Para trabajar con streams hay que tener en cuenta varias cosas:

- Vamos a utilizar 4 streams.
- Hay que partir el algoritmo en 4 tareas (con N/4 elementos por tarea) y asociar cada tarea a un stream diferente.
- En el código resultante habrá:
 - 4 transferencias de partes del vector A, en streams diferentes

- 4 transferencias de partes del vector B, en streams diferentes
 - 4 ejecuciones del kernel, en streams diferentes
 - 4 transferencias de partes del vector C, en streams diferentes
 - Para trabajar con diferentes partes de los vectores, sólo hay que jugar con los punteros.
- El kernel no se ha de modificar.
 - La forma de emitir las diferentes llamadas influye en el rendimiento, os aconsejamos que probéis diversas soluciones.

6.4 Kernel 02 (stream02.cu)

La forma que os hemos propuesto para utilizar streams es demasiado simplista. Hemos partido una XXX (podéis llamar esta operación como más os guste) de vectores con N elementos en 4 XXX de vectores de N/4 elementos.

Una solución más útil, pensando en problemas futuros, sería partir el problema en tareas con un número fijo de elementos (por ejemplo, si $N=50000 \cdot 1024$, podríamos usar $N_k=2500 \cdot 1024$) y enviar a ejecutar las operaciones que sean necesarias con tamaño N_k .

Modificad el código anterior, teniendo en cuenta que enviaremos kernels con $N_k=2500 \cdot 1024$ (por ejemplo). Recordad que estabamos utilizando 4 streams. Ahora utilizaremos más, por ejemplo 8, lo haremos utilizando un vector de streams como éste:

```
cudaStream_t stream[8];
```

En definitiva, la idea es emitir las diferentes llamadas al kernel, con las transferencias necesarias, utilizando un bucle. Obviamente, cada grupo de transferencias-kernel hay que emitirlo a un stream diferente:

```
for (---, str=0; ---; ---, str = (str+1)%nStreams) {
    . . .
    Kernel00<<< ---, stream[str]>>>( --- );
    . . .
}
```

6.5 Algunos comentarios respecto a la toma de tiempos

Para la toma de tiempos utilizamos el mismo método que en sesiones anteriores:

```
cudaEventRecord(E0, 0);
//
// CODIGO A EVALUAR
```

```
//  
cudaEventRecord(E3, 0); cudaEventSynchronize(E3);  
cudaEventElapsedTime(&TiempoTotal, E0, E3);
```

Al acabar, en la variable `TiempoTotal`, tenemos el tiempo de ejecución de nuestro código.

Hay que puntualizar que el segundo parámetro de la función `cudaEventRecord()` indica el stream en el que se registra el evento. Pero, según el manual de **CUDA**: “... events in stream zero are completed after all preceding tasks and commands in all streams are completed ...” (¡Perfecto!).

Por último, los tiempos de ejecución no siempre son coherentes. Para las pruebas os aconsejamos que sigáis el siguiente protocolo. Para probar que los algoritmos funcionan, utilizad tamaños pequeños:

```
./kernel01.exe 10000 Y  
./kernel02.exe 10000 Y
```

Para tomar tiempos, utilizad tamaños más grandes y desactivad la comprobación del resultado:

```
./kernel01.exe 300000 N  
./kernel02.exe 300000 N
```

6.6 Consideraciones Finales

Igual que hicimos en sesiones anteriores, para facilitaros las cosas, y sólo para evitar que os quedéis encallados, hemos incluido un directorio en el que están todos los kernels solucionados. En algunas de las soluciones se incluyen diferentes formas de lanzar los streams.

Con esto se acaban las sesiones dirigidas. A partir de ahora dedicaremos las sesiones de laboratorio a vuestro proyecto. Algunas consideraciones respecto al proyecto:

- Se realiza en grupos de 2 estudiantes.
- El proyecto os ha de costar unas 20-30 horas de trabajo, teniendo en cuenta que nos quedan varias clases de laboratorio.
- En la entrega del proyecto, tenéis que hacer una pequeña memoria (8-10 páginas) y entregar el código del mismo.
- Contactaremos con cada grupo para hacer un seguimiento del proyecto.

7 PROYECTO – ALGORITMO DE STRASSEN

El algoritmo de Strassen es un algoritmo utilizado para el producto de matrices. No es el algoritmo más rápido conocido, pero es más rápido que el algoritmo convencional, y es útil en la práctica para matrices grandes.

7.1 Introducción

El algoritmo fue publicado en 1969 por Volker Strassen [1]. Sólo es ligeramente más rápido que el algoritmo estándar para multiplicar matrices, pero fue el primero en mostrar que el enfoque estándar no es óptimo. Si el algoritmo original tiene un coste $O(n^3)$, el algoritmo de Strassen tiene un coste $O(N^{\log_2 7 + o(1)}) \approx O(N^{2,807})$.

En la actualidad los mejores algoritmos para el producto de matrices tienen un rendimiento asintótico de $O(N^{2,3728639})$ [2]. Aunque a diferencia del algoritmo de Strassen no se suelen utilizar en la práctica porque sólo consiguen esos rendimientos con matrices muy grandes.

7.2 El algoritmo

Por simplicidad, el algoritmo trabaja con matrices cuadradas y con sus dimensiones potencia de 2. Es decir, queremos hacer la siguiente operación: $C(2^n \times 2^n) \leftarrow A(2^n \times 2^n) \cdot B(2^n \times 2^n)$

Estas 3 matrices pueden partirse en submatrices del mismo tamaño de la siguiente forma:

$$A = \begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix}, \quad B = \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix}, \quad C = \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Siendo todas las submatrices $A_{i,j}$, $B_{i,j}$ y $C_{i,j}$ de dimensiones $2^{n-1} \times 2^{n-1}$, el producto de matrices estándar sería:

- $C_{1,1} = A_{1,1} \cdot B_{1,1} + A_{1,2} \cdot B_{2,1}$
- $C_{1,2} = A_{1,1} \cdot B_{1,2} + A_{1,2} \cdot B_{2,2}$
- $C_{2,1} = A_{2,1} \cdot B_{1,1} + A_{2,2} \cdot B_{2,1}$
- $C_{2,2} = A_{2,1} \cdot B_{1,2} + A_{2,2} \cdot B_{2,2}$

Evaluando el algoritmo se realizan 8 productos de matrices $O((2^{n-1})^3)$, que equivale al coste que ya conocíamos $O((2^n)^3)$.

A partir de aquí empieza el algoritmo de Strassen. Primero hemos de calcular estas siete matrices:

- $M_1 = (A_{1,1} + A_{2,2}) \cdot (B_{1,1} + B_{2,2})$

- $M_2 = (A_{2,1} + A_{2,2}) \cdot B_{1,1}$
- $M_3 = A_{1,1} \cdot (B_{1,2} - B_{2,2})$
- $M_4 = A_{2,2} \cdot (B_{2,1} - B_{1,1})$
- $M_5 = (A_{1,1} + A_{1,2}) \cdot B_{2,2}$
- $M_6 = (A_{2,1} - A_{1,1}) \cdot (B_{1,1} + B_{1,2})$
- $M_7 = (A_{1,2} - A_{2,2}) \cdot (B_{2,1} + B_{2,2})$

Usando las matrices M_i , ya podemos calcular el producto de matrices de la siguiente forma:

- $C_{1,1} = M_1 + M_4 - M_5 + M_7$
- $C_{1,2} = M_3 + M_5$
- $C_{2,1} = M_2 + M_4$
- $C_{2,2} = M_1 - M_2 + M_3 + M_6$

El algoritmo se vuelve a aplicar de forma recursiva para calcular cada una de las submatrices $C_{i,j}$.

Evaluar el coste del algoritmo queda al margen de los objetivos de la asignatura, pero puede verse fácilmente que, ya en la primera iteración, sólo hacemos 7 productos de matrices $O((2^{n-1})^3)$, a costa de hacer unas cuantas sumas matriciales con un coste mucho menor $O((2^{n-1})^2)$.

7.3 Implementación CUDA en 1 GPU

La primera parte del proyecto es la implementación del algoritmo de Strassen para multiplicar matrices en 1 GPU. Podemos implementar el algoritmo siguiendo las siguientes pautas:

- Trabajaremos con matrices cuadradas y dimensiones potencia de 2 (2^n). Probaremos con todas las matrices cuadradas con dimensiones potencia de 2 entre 128×128 y 8192×8192 .
- Como algoritmo para calcular el producto podéis utilizar el producto a bloques de la Sesión 4. En particular el producto que funcionaba para dimensiones múltiplo del número de *threads*.
- No implementaremos el algoritmo recursivo. Sólo calcularemos una vez las matrices M_i de dimensiones $(2^{n-1}) \times (2^{n-1})$. Os aconsejamos que defináis un kernel diferente para cada matriz M_i .
- No es necesario, pero os aconsejamos particionar las matrices.
- alguna de estas pautas os serán muy útiles para las versiones multiGPU.
- Cuando comparéis los tiempos de ejecución con la versión a bloques, comparad sólo los tiempos de ejecución de los kernels.

7.4 Implementación CUDA en varias GPUs

En este apartado os pedimos que hagáis 2 versiones diferentes:

- Con 2 GPUs, en este caso os proponemos que:
 - la GPU0 calcule $C_{1,1}$ y $C_{1,2}$,
 - la GPU1 calcule $C_{2,1}$ y $C_{2,2}$.
- Con 4 GPUs, en este caso cada GPU ha de calcular una de las submatrices $C_{i,j}$.

En ambas versiones se pueden estudiar varias cosas:

- **Equilibrio de carga.** ¿Cómo distribuir la carga de trabajo entre las diferentes GPUs para que no haya ninguna inactiva demasiado tiempo?
- **Replicar cálculos vs calcular y transmitir.** Todas las matrices M_i se utilizan 2 veces. Algunas de ellas se necesitarán en 2 GPUs diferentes. Hay que evaluar que es más efectivo: calcularlas en ambas GPUs o calcularla en 1 y enviarla a la otra.
- **Comunicación GPU - GPU.** Hay que programar movimiento de datos entre GPUs sin pasar por la CPU. Este movimiento de datos no lo hemos visto en clase y tendréis que buscar en el manual de **CUDA** cómo hacerlo.
- **Tiempos de ejecución.** Obtener los tiempos de ejecución en una aplicación multiGPU no es trivial. ¡Hacedlo con cuidado!

Para ayudaros en la implementación multiGPU os hemos incluido tres tablas en las que indicamos que matrices son necesarias para calcular las matrices M_i y $C_{i,j}$. Os ayudará a saber cómo distribuir los cálculos y los datos entre las GPUs. Uno de los objetivos de vuestra implementación debería ser ocupar la menor cantidad de memoria posible. Porque os vamos a pedir que la versión con dos GPUs sea capaz de calcular el producto de matrices de dimensiones 32768×32768 ($2^{15} \times 2^{15}$), y la versión con 4 GPUs matrices de dimensiones 65536×65536 ($2^{16} \times 2^{16}$).

—	$A_{1,1}$	$A_{1,2}$	$A_{2,1}$	$A_{2,2}$	$B_{1,1}$	$B_{1,2}$	$B_{2,1}$	$B_{2,2}$
M_1	✓	-	-	✓	✓	-	-	✓
M_2	-	-	✓	✓	✓	-	-	-
M_3	✓	-	-	-	-	✓	-	✓
M_4	-	-	-	✓	✓	-	✓	-
M_5	✓	✓	-	-	-	-	-	✓
M_6	✓	-	✓	-	✓	✓	-	-
M_7	-	✓	-	✓	-	-	✓	✓

Cuadro 1: MATRICES $A_{I,J}$ Y $B_{I,J}$ NECESARIAS PARA CALCULAR LAS MATRICES M_I .

—	M_1	M_2	M_3	M_4	M_5	M_6	M_7
$C_{1,1}$	✓	-	-	✓	✓	-	✓
$C_{1,2}$	-	-	✓	-	✓	-	-
$C_{2,1}$	-	✓	-	✓	-	-	-
$C_{2,2}$	✓	✓	✓	-	-	✓	-

Cuadro 2: MATRICES M_I NECESARIAS PARA CALCULAR LAS MATRICES $C_{I,J}$.

—	$A_{1,1}$	$A_{1,2}$	$A_{2,1}$	$A_{2,2}$	$B_{1,1}$	$B_{1,2}$	$B_{2,1}$	$B_{2,2}$
$C_{1,1}$	✓	✓	-	✓	✓	-	✓	✓
$C_{1,2}$	✓	✓	-	-	-	✓	-	✓
$C_{2,1}$	-	-	✓	✓	✓	-	✓	-
$C_{2,2}$	✓	-	✓	✓	✓	✓	-	✓

Cuadro 3: MATRICES $A_{I,J}$ Y $B_{I,J}$ NECESARIAS PARA CALCULAR LAS MATRICES $C_{I,J}$.

7.5 Resultados esperados

Es importante que comprobéis que la aplicación funciona de forma similar a cómo lo hacíamos en las sesiones dirigidas. No es necesario que lo comprobéis en todas las dimensiones, con que lo hagáis en dos dimensiones (128×128 y 2048×2048) es suficiente.

En la presentación de los rendimientos sólo nos interesa el rendimiento de los kernels. Todo lo relativo a partir las matrices A y B en las submatrices $A_{i,j}$ y $B_{i,j}$, así como las comunicaciones entre CPU y GPU no lo tendremos en cuenta. Como resultado nos gustaría dos tablas una con tiempos de ejecución y otra con rendimientos (GFLOPs) para todas las versiones: a bloques, Strassen en 1 GPU, Strassen en 2 GPUs y Strassen en 4 GPUs.

Las tamaños de las matrices deberían empezar en 128×128 y acabar en el máximo tamaño que podáis ejecutar. En una K40, hemos comprobado que se puede ejecutar un producto de matrices de 8192×8192 . En este caso no se puede comprobar el resultado porque el tiempo de ejecución secuencial en la CPU es enorme. Pero si se pueden calcular unos cuantos resultados parciales, simplemente cambiando los bucles i y j del test:

```
int TestMM(int N, int M, int P, float *A, float *B, float *C) {
    ...
    for (i=0; i<N; i=i+17)
        for (j=0; j<M; j=j+19)
            comprobar C[i][j]
    ...
}
```

No podemos asegurar que funcione al 100 % pero si hay algún error debería aparecer.

8 PROYECTO – HISTOGRAM EQUALIZATION

El algoritmo de equalización es un algoritmo utilizado para aumentar el contraste de las siluetas u objetos que aparecen en una imagen con poca definición, mucho brillo, etc.

8.1 Introducción

Figura 1 muestra un ejemplo de la equalización de la imagen en blanco y negro de la izquierda, para obtener la imagen de la derecha. La equalización de una imagen puede ayudar, por ejemplo, a mejorar el contraste en imágenes médicas.

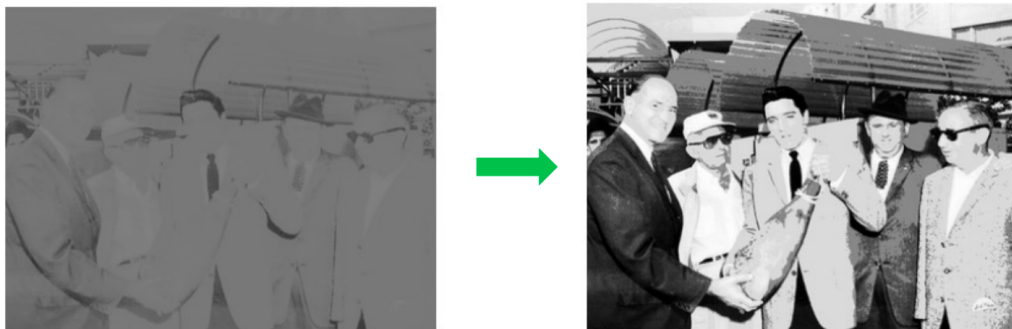


Figura 1: EJEMPLO DE EQUALIZACIÓN DE UNA IMAGEN.

8.2 Algoritmo

El algoritmo básicamente consiste en:

- Generar un histograma de la imagen
- A partir del histograma, en función de los valores máximos y mínimos, generar una tabla de traducción (equalización).
- Equalizar la imagen.

8.2.1 Tabla de Equalización

Para generar la tabla de equalización, puede ser tan simple como pasar de una distribución $[min : max]$ a otra $[0 : N - 1]$. Figura 2 muestra un ejemplo de como sería una tabla de traducción. Si tenemos una imagen en blanco y negro, cada píxel es un valor entre 0 y 255, el algoritmo es simple de implementar.

Si trabajamos con RGB (24 bits) el algoritmo puede tener muchas variantes. Podríamos hacer un histograma para cada color y equalizar cada color por separado. También podríamos tomar la parte alta de cada color, por ejemplo los 4 bits de mayor peso de cada color y hacer el histograma con los 12 bits resultantes (ver Figura 3).

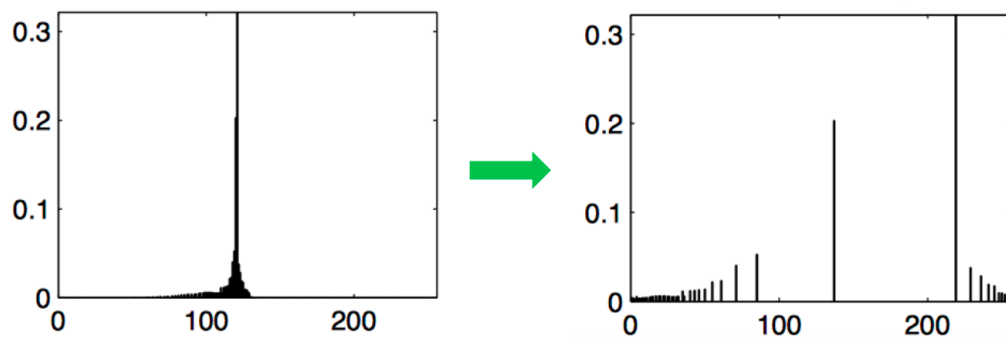


Figura 2: EJEMPLO DE UNA TABLA DE EQUALIZACIÓN.

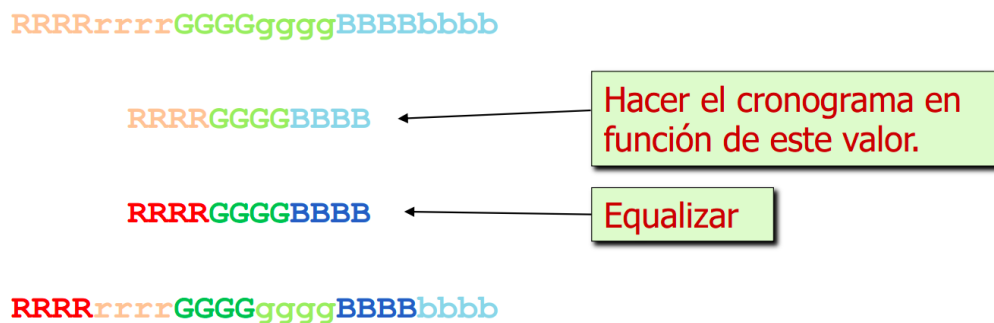


Figura 3: EJEMPLO DE EQUALIZACIÓN DE UNA IMAGEN CON COLOR.

8.3 Implementación CUDA en 1 GPU

La primera parte del proyecto es la implementación del algoritmo de equalización en 1 GPU.

- Trabajaremos con diferentes imágenes de diferentes tamaños.
- Podéis utilizar la versión de CPU que os subministraremos.
- Os sugerimos realizar diferentes versiones del programa para que::
 - El kernel trabaje por filas, columnas, o a bloques.
 - Utilizar o no shared memory.
 - El programa use memoria pinned o no.
 - El programa trabaje con diferentes streams.

8.4 Implementación CUDA en varias GPUs

En este apartado os pedimos que hagáis una versión que trabaje con 1, 2, 4 GPUs, para la mejor versión del apartado anterior, sin usar streams.

8.5 Resultados esperados

Es importante que comprobéis que la aplicación funciona de forma similar a cómo lo hacíamos en las sesiones dirigidas.

En la presentación de los rendimientos sólo nos interesa el rendimiento de los kernels. Como resultado nos gustaría una tabla con tiempos de ejecución y una gráfica donde se muestre el speedup conseguido respecto a la versión CPU.

9 PROYECTO – HEAT: JACOBI O GAUSS-SEIDEL

Jacobi y Gauss-Seidel son dos estrategias de solución del `heat equation` utilizando un algoritmo de tipo Stencil.

9.1 Introducción

El `heat equation` calcula la propagación de varios focos de calor. Figura 4 muestra un ejemplo de la propagación de calor con dos focos de calor en una matriz bidimensional. Las soluciones encontradas cuando se usa Jacobi y Gauss-Seidel no tienen por qué ser exactamente la misma, pero ambas son correctas. En cualquier caso, el programa itera sobre la matriz inicial hasta que converge a una solución. Figura 5 muestra un ejemplo de programa principal, que llama a una solución u otra en función de los parámetros de entrada. Como se puede observar, en la solución Jacobi se necesitan dos matrices, que se van copiando la una en la otra en cada iteración. En la solución Gauss-Seidel, sólo se necesita una matriz y no hay copia.

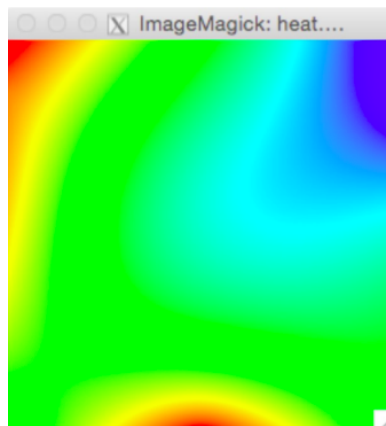


Figura 4: EJEMPLO DEL HEAT CON DOS FOCOS DE CALOR.

9.2 Algoritmo Jacobi

Algoritmo iterativo que procesa una matriz bidimensional y deja el valor en otra matrix. El programa principal llama a el `solver Jacobi` hasta que la solución converge. A cada iteración del programa, se hace un intercambio de las dos matrices.

Figura 6 muestra un ejemplo de implementación secuencial en la CPU del Jacobi. Como se puede observar lee de una matriz de entrada varios elementos vecinos y el elemento (i,j) para actualizar el elemento de la matriz auxiliar.

9.3 Algoritmo Gauss-Seidel

Algoritmo iterativo que procesa una matriz bidimensional y deja el valor en la misma matrix. El programa principal llama a el `solver Gauss-Seidel` hasta que la solución converge.

```

// starting time
runtime = wtime();

iter = 0;
while(1) {
    switch( param.algorithm ) {
        case 0: // JACOBI
            residual = relax_jacobi(param.u, param.uhelp, np, np);
            // Copy uhelp into u
            copy_mat(param.uhelp, param.u, np, np);
            break;
        case 1: // GAUSS
            residual = relax_gauss(param.u, np, np);
            break;
    }

    iter++;

    // solution good enough ?
    if (residual < 0.00005) break;

    // max. iteration reached ? (no limit with maxiter=0)
    if (param.maxiter>0 && iter>=param.maxiter) break;
}

```

Figura 5: EJEMPLO DEL PROGRAMA PRINCIPAL DEL HEAT.

```

/*
 * Blocked Jacobi solver: one iteration step
 */
double relax_jacobi (double *u, double *utmp, unsigned sizex, unsigned sizey)
{
    double diff, sum=0.0;

    int howmany=4;
    for (int blockid = 0; blockid < howmany; ++blockid) {
        int i_start = lowerb(blockid, howmany, sizex);
        int i_end = upperb(blockid, howmany, sizex);
        for (int i=max(1, i_start); i<= min(sizex-2, i_end); i++) {
            for (int j=1; j<= sizey-2; j++) {
                utmp[i*sizey+j]= 0.25 * ( u[ i*sizey      + (j-1) ]+ // left
                                           u[ i*sizey      + (j+1) ]+ // right
                                           u[ (i-1)*sizey + j      ]+ // top
                                           u[ (i+1)*sizey + j      ]); // bottom

                diff = utmp[i*sizey+j] - u[i*sizey + j];
                sum += diff * diff;
            }
        }
    }

    return sum;
}

```

Figura 6: EJEMPLO DE CÓDIGO DE LA SOLUCIÓN JACOBI.

Figura 7 muestra un ejemplo de implementación secuencial en la CPU del Gauss-Seidel. A diferencia el Jacobi, esta solución no precisa de una matriz auxiliar y actualiza los elementos de la misma matriz. En principio esto ayuda a converger más rápido a una solución, pero provoca una dependencia entre iteraciones.

9.4 Implementación CUDA en 1 GPU

La primera parte del proyecto es la implementación del algoritmo de Heat usando Jacobi o Gauss-Seidel en 1 GPU. La versión de Gauss-Seidel es más complicada porque hay que tener en cuenta que hay

```

/*
 * Blocked Gauss-Seidel solver: one iteration step
 */
double relax_gauss (double *u, unsigned sizex, unsigned sizey)
{
    double unew, diff, sum=0.0;

    int howmany=4;
    for (int blockid = 0; blockid < howmany; ++blockid) {
        int i_start = lowerb(blockid, howmany, sizex);
        int i_end = upperb(blockid, howmany, sizex);
        for (int i=max(1, i_start); i<= min(sizex-2, i_end); i++) {
            for (int j=1; j<= sizey-2; j++) {
                unew= 0.25 * ( u[ i*sizey + (j-1) ]+ // left
                             u[ i*sizey + (j+1) ]+ // right
                             u[ (i-1)*sizey + j ]+ // top
                             u[ (i+1)*sizey + j ]); // bottom

                diff = unew - u[i*sizey+ j];
                sum += diff * diff;
                u[i*sizey+j]=unew;
            }
        }
    }

    return sum;
}

```

Figura 7: EJEMPLO DE SOLUCIÓN GAUSS-SEIDEL.

dependencias entre iteraciones. Por consiguiente, el número de diferentes tamaños a probar, y los experimentos pueden reducirse. Por ejemplo, dejar de hacer alguna de las versiones del kernel que os comentamos, pero realizando al menos la de bloques.

- Os sugerimos que probéis con diferentes tamaños de matriz y números de focos.
- Podéis utilizar la versión de CPU que os subministraremos.
- Os sugerimos realizar diferentes versiones del programa para que::
 - El kernel trabaje por filas, columnas, o a bloques.
 - Utilitzar o no shared memory.
 - El programa use memoria pinned o no.
 - El programa trabaje con diferentes streams.

9.5 Implementación CUDA en varias GPUs

En este apartado os pedimos que hagáis una versión que trabaje con 1, 2, 4 GPUs, para la mejor versión del apartado anterior, sin usar streams.

9.6 Resultados esperados

Es importante que comprobéis que la aplicación funciona de forma similar a cómo lo hacíamos en las sesiones dirigidas.

En la presentación de los rendimientos sólo nos interesa el rendimiento de los kernels. Como resultado nos gustaría una tabla con tiempos de ejecución y una gráfica donde se muestre el speedup conseguido respecto a la versión CPU.

10 PROYECTO – MANDELBROT

El programa de Mandelbrot seguramente lo habéis trabajado ya en la asignatura de Paralelismo y lo habéis paralelizado con OpenMP. El programa se usa para calcular el conjunto de Mandelbrot.

10.1 Introducción

El programa que se utilizará es el cálculo del *conjunto de Mandelbrot*, un conjunto particular de puntos, en el dominio complejo, cuyo límite genera una forma fractal bidimensional distintiva y fácilmente reconocible (Figura 8).

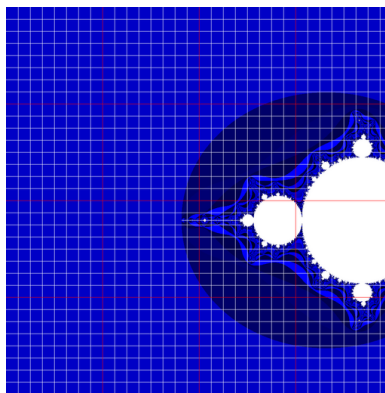


Figura 8: FORMA FRACTAL.

10.2 Algoritmo de Mandelbrot

Para cada punto c en un espacio bidimensional delimitado, se aplica iterativamente la recurrencia polinómica cuadrática compleja $z_{n+1} = z_n^2 + c$ para determinar si pertenece o no al conjunto de Mandelbrot. El punto forma parte del conjunto de Mandelbrot si, al partir de $z_0 = 0$ y aplicar la iteración repetidamente, el valor absoluto de z_n nunca supera un número determinado, independientemente del valor de n .

Se crea un gráfico del conjunto de Mandelbrot coloreando cada punto c en el plano complejo con el número de pasos max para el cual $|z_{max}| \geq 2$ (o simplemente $|z_{max}|^2 \geq 2 * 2$ para evitar el cálculo de la raíz cuadrada en el módulo de un número complejo). Para que el problema sea factible, el número máximo de pasos también es limitado: si se alcanza ese número, se dice que el punto c pertenece al conjunto de Mandelbrot.

Si quieres saber más sobre el conjunto de Mandelbrot, te recomendamos que eches un vistazo a la siguiente página de Wikipedia:

http://en.wikipedia.org/wiki/Mandelbrot_set⁸

⁸Website last visited on March 13, 2025

En la sección "Computer drawings" de esa página, encontrará diferentes maneras de representar el conjunto de Mandelbrot. En particular, utilizaremos la idea del algoritmo de Mariani[3] para comprobar el borde de los rectángulos (*tiles*) y evitar el cálculo de *tiles* completos, ya que sabemos que todos los puntos tienen el mismo valor. La figura 8 muestra una versión en mosaico de la imagen de Mandelbrot. Los mosaicos que caen en un área con el mismo color se pueden calcular copiando únicamente el mismo valor.

10.3 Implementación CUDA en 1 GPU

La primera parte del proyecto es la implementación del algoritmo de Mandelbrot en 1 GPU.

- Os sugerimos que probéis con diferentes tamaños de matriz, cuadradas o no.
- Podéis utilizar la versión de CPU que os subministraremos como base.
- Os sugerimos realizar diferentes versiones del programa para que::
 - El kernel trabaje por filas, columnas, o a bloques.
 - Utilizar o no shared memory.
 - El programa use memoria pinned o no.
 - El programa trabaje con diferentes streams.

10.4 Implementación CUDA usando Paralelismo dinámico

En este apartado os pedimos que hagáis una versión que trabaje con 1 GPU pero que use paralelismo dinámico. Os proporcionaremos un ejemplo de como explotar este paralelismo, pero se parece un poco a utilizar `single` en OpenMP con tal de que este thread cree más "tareas" a realizar en la GPU.

10.5 Resultados esperados

Es importante que comprobéis que la aplicación funciona de forma similar a cómo lo hacíamos en las sesiones dirigidas.

En la presentación de los rendimientos sólo nos interesa el rendimiento de los kernels. Como resultado nos gustaría una tabla con tiempos de ejecución y una gráfica donde se muestre el speedup conseguido respecto a la versión CPU.

11 PYTHON Y CUDA

Todos los ejemplos que hemos utilizado en clase y laboratorio utilizaban C y C++ como lenguaje de programación. Os proponemos utilizar como Lenguaje de programación Python.

11.1 Objetivo del proyecto

El objetivo del proyecto es replicar algunas de las sesiones dirigidas de laboratorio que ya hemos hecho. Esas sesiones estaban programadas en C, ahora queremos que las programéis en Python.

De las sesiones 3, 4, 5 y 6. Habría que programar al menos 3 de ellas, o 4 de ellas parcialmente.

Una vez tengamos la versión Python, para la versión C podéis usar las vuestras o las soluciones que os dimos, os pediríamos que hicieráis una pequeña evaluación:

- Medir los rendimientos de cada aplicación, utilizando las métricas más adecuadas, y comparar los resultados de C y Python. Si es posible, tenéis que probar con diferentes tamaños de problema (por ejemplo, en la Sesión 04).
- Evaluar la complejidad de desarrollo: cantidad de líneas de código, facilidad de depuración y mantenimiento, ...

En la memoria, también tenéis que incluir una pequeña explicación de cómo se usa **CUDA** en Python, teniendo en cuenta que Python es un lenguaje interpretado y a la GPU, le ha de llegar código compilado.

11.2 Punto de Partida

Para empezar a trabajar os daremos un tar, similar al de las sesiones dirigidas, en donde hay unos cuantos ejemplos de Python utilizando **CUDA** (muy simples), y el `job.sh` necesario para lanzar un programa Python a las colas de boada.

Bibliografía

- [1] Volker Strassen. Gaussian elimination is not optimal. *Numer. Math.*, 13(4):354–356, August 1969.
- [2] François Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, ISSAC '14, pages 296–303, New York, NY, USA, 2014. ACM.
- [3] A. K. Dewdney. Computer recreations: A tour of the mandelbrot set aboard the mandelbus. *Scientific American*, page 111, February 1989. JSTOR 24987149.